

LLSMS2012 - Macro Finance

Individual Work

Nicolas Poncin - 76152100

Professor

Leonardo Iania

2025–2026

Contents

1 Time series **2**

1.1 AR(1) Process 2

1.2 Mean and variance of long-horizon returns 10

2 Stock Market **18**

2.1 Efficient Frontier 18

2.2 Stock returns and strategies 27

3 Bond Market **31**

3.1 Test 1: Long yield changes over the life of the short bond 31

3.2 Test 2: Short yield changes over the life of the long bond 33

4 AI assistance **35**

5 Appendices **36**

1 Time series

1.1 AR(1) Process

A stock price x_t follows an AR(1) process, i.e.

$$x_t = \rho x_{t-1} + \varepsilon_t$$

where ε_t are a sequence of i.i.d. normally distributed random variables with mean 0 and standard deviation σ_ε.

a) Conditional means

We know: $\mathbf{E}_t(\varepsilon_{t+j}) = 0$.

$$\begin{aligned}
\mathbf{E}_t(x_{t+1}) &= \mathbf{E}_t(\rho x_t + \varepsilon_{t+1}) \\
&= \rho \underbrace{\mathbf{E}_t(x_t)}_{=x_t} + \mathbf{E}_t(\varepsilon_{t+1}) \\
&= \rho x_t + 0
\end{aligned}$$

$$\begin{aligned}
\mathbf{E}_t(x_{t+2}) &= \mathbf{E}_t\left(\rho x_{t+1} + \varepsilon_{t+2}\right) \\
&= \rho\, \mathbf{E}_t(x_{t+1}) + \mathbf{E}_t(\varepsilon_{t+2}) \\
&= \rho \cdot \rho x_t + 0 \\
&= \rho^2 x_t
\end{aligned}$$

The sequence of conditional mean becomes:

$$\boxed{\mathbf{E}_t(x_{t+j}) = \rho^j \cdot x_t}$$

Interpretation: An AR(1) process cannot see into the future; the best guess for the j-th iterate is the current iterate x_t scaled by ρ^j.

b) Conditional Variances

We know: $\mathrm{Var}(\varepsilon_{t+j}) = \sigma_\varepsilon^2$, $\mathrm{Cov}(\varepsilon_{t+j}, \varepsilon_{t+k}) = 0$ for $j \neq k$.

$$\begin{aligned}
\mathrm{Var}_t(x_{t+1}) &= \mathrm{Var}_t(\rho x_t + \varepsilon_{t+1}) \\
&= \mathrm{Var}_t(\rho x_t) + \mathrm{Var}_t(\varepsilon_{t+1}) + 2\,\mathrm{Cov}(\rho x_t,\, \varepsilon_{t+1}) \\
&= \rho^2 \underbrace{\mathrm{Var}_t(x_t)}_{=0 \text{ as } x_t \text{ is already observed at time } t} + \sigma_\varepsilon^2 + 0 \\
&= \sigma_\varepsilon^2
\end{aligned}$$

$$\begin{aligned}
\mathrm{Var}_t(x_{t+2}) &= \mathrm{Var}_t(\rho x_{t+1} + \varepsilon_{t+2}) \\
&= \rho^2\,\mathrm{Var}_t(x_{t+1}) + \sigma_\varepsilon^2 + 0 \\
&= \rho^2\sigma_\varepsilon^2 + \sigma_\varepsilon^2 \\
&= \sigma_\varepsilon^2(1+\rho^2)
\end{aligned}$$

By induction, expanding $x_{t+j} = \rho^j x_t + \sum_{i=1}^{j} \rho^{j-i}\varepsilon_{t+i}$, and since the shocks are i.i.d. and independent of x_t:

$$\begin{aligned}
\mathrm{Var}_t(x_{t+j}) &= \mathrm{Var}_t\left(\sum_{i=1}^{j} \rho^{j-i}\varepsilon_{t+i}\right) \\
&= \sum_{i=1}^{j} \rho^{2(j-i)}\,\sigma_\varepsilon^2 \\
&= \sigma_\varepsilon^2 \sum_{k=0}^{j-1} \rho^{2k} \\
&= \sigma_\varepsilon^2 \cdot \frac{1-\rho^{2j}}{1-\rho^2}
\end{aligned}$$

$$\boxed{\mathrm{Var}_t(x_{t+j}) = \sigma_\varepsilon^2 \cdot \frac{1-\rho^{2j}}{1-\rho^2}}$$

Interpretation: Because x_t is known at time t, the only source of uncertainty comes from the future shocks $\varepsilon_{t+1}, \ldots, \varepsilon_{t+j}$. Therefore, the conditional variance increases with the horizon, with each shock contributing less and less due to the geometric decay factor ρ^2.

c) Plots

i) Unconditional mean plot

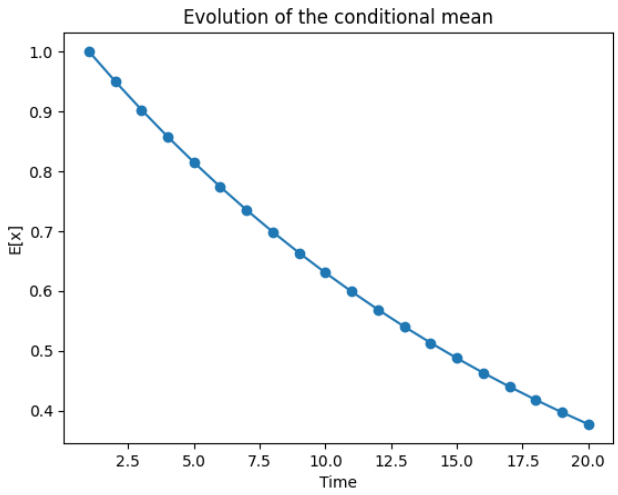


Figure 1: Evolution of the conditional mean

ii) Adding a ± 1 conditional standard deviation

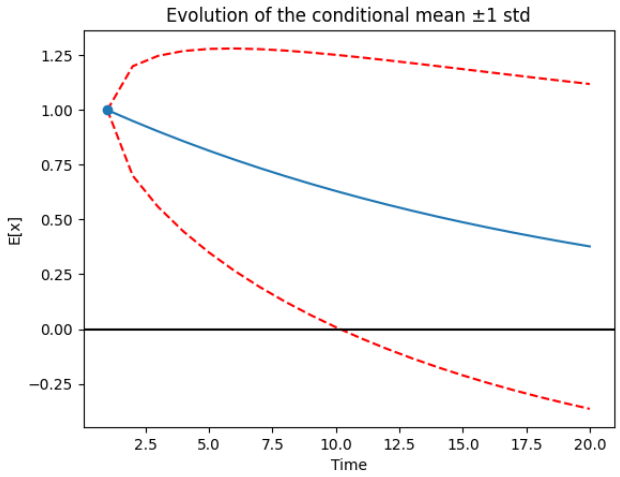


Figure 2: Evolution of the conditional mean ± 1 std

The conditional mean declines gradually toward the long-run mean. Indicating that the best guess for future iteration is the long-run mean. Moreover, the conditional standard deviation band widens as the forecast horizon increases, showing that uncertainty accumulates over time even though the best forecast continues to drift back toward the unconditional mean.
In the following charts, a larger initial value, such as t=100, is used to observe mean-reverting behavior, since the conditional mean then drifts toward the unconditional mean.

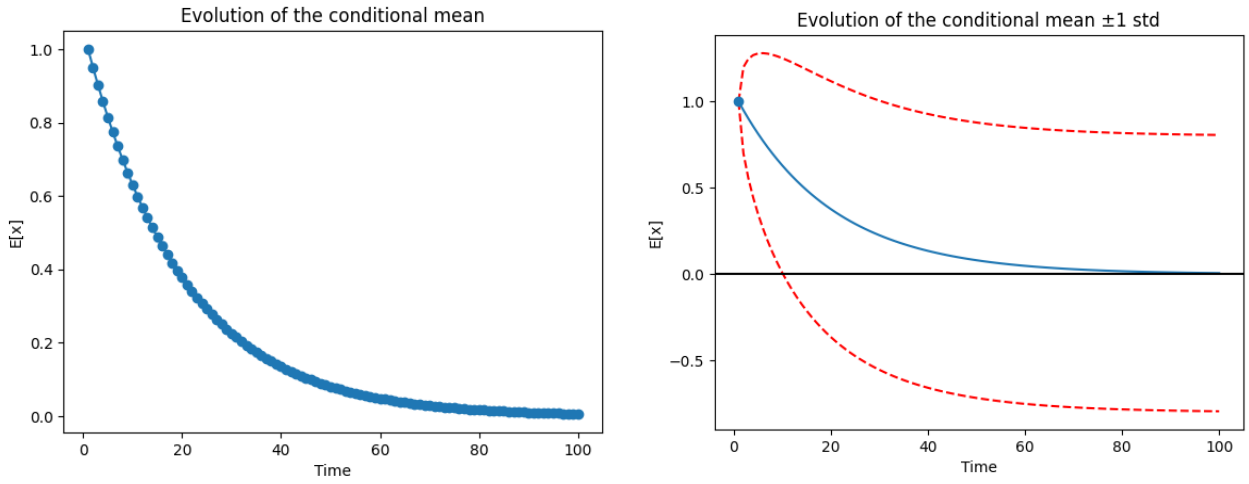


Figure 3: Evolution of the conditional mean with $t = 100$

iii) Chebyshev inequality vs Normality assumption

We first compute the conditional mean and variance of the future value x_{t+h} given the information available at time t. We then construct prediction cones using two approaches: a distribution-free bound based on Chebyshev's inequality and a tighter bound based on the assumption that the shocks are Gaussian.

1. Chebyshev's inequality provides a conservative bound that does not require normality. For any random variable X with mean μ and variance σ^2,

$$\Pr\left(|X - \mu| \geq k\sigma\right) \leq \frac{1}{k^2},$$

so

$$\Pr\left(|X - \mu| < k\sigma\right) \geq 1 - \frac{1}{k^2}.$$

Therefore, the 90% and 95% prediction bands are obtained by setting $k = \sqrt{10}$ and $k = \sqrt{20}$, respectively.

2. Under normalilty, the prediction bands are narrower. If $x_{t+h} \mid \mathcal{F}_t$ is Gaussian with conditional mean $E_t[x_{t+h}]$ and conditional standard deviation $\sqrt{\mathrm{Var}_t(x_{t+h})}$, then the 90% and 95% bands are

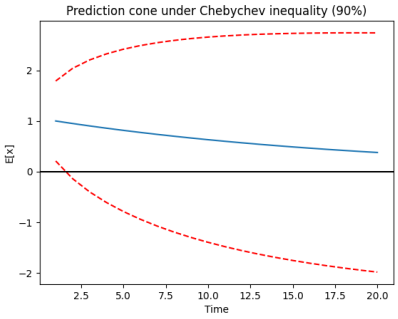
$$E_t[x_{t+h}] \pm 1.645 \sqrt{\mathrm{Var}_t(x_{t+h})}$$

and

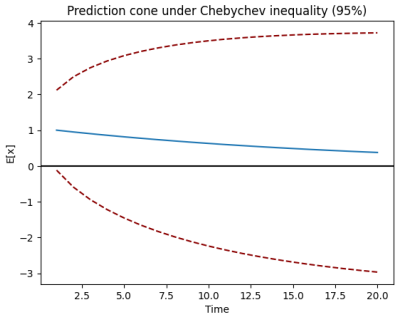
$$E_t[x_{t+h}] \pm 1.96 \sqrt{\mathrm{Var}_t(x_{t+h})}.$$

To answer the question, we compute the conditional mean and variance for horizons $h = 1, \ldots, 20$, plot the two sets of prediction cones, and compare them.

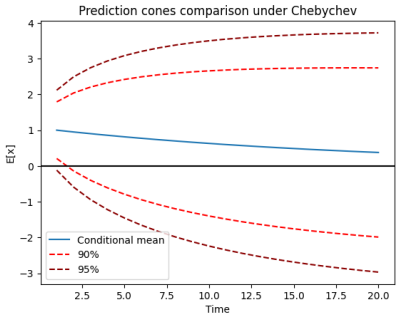
Figure 4: Prediction cone visualization under Chebyshev inequality



(a) 90% bands

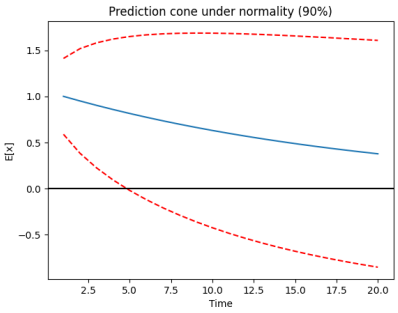


(b) 95% bands

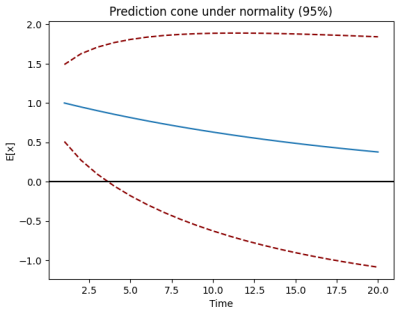


(c) 90% and 95% comparison

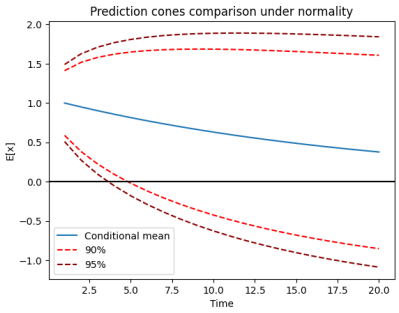
Figure 5: Prediction cone visualization under normality assumption



(a) 90% bands

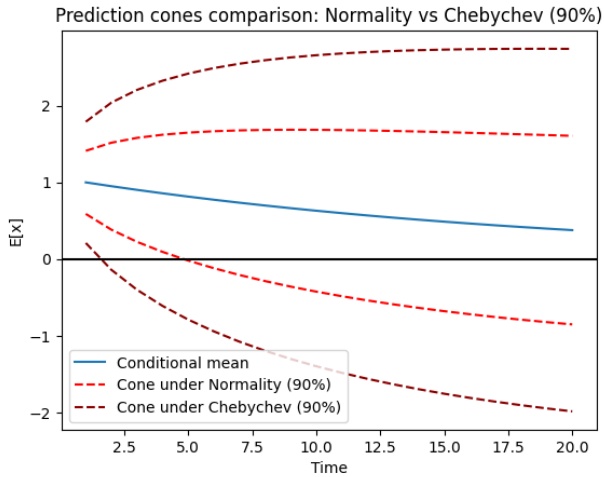


(b) 95% bands

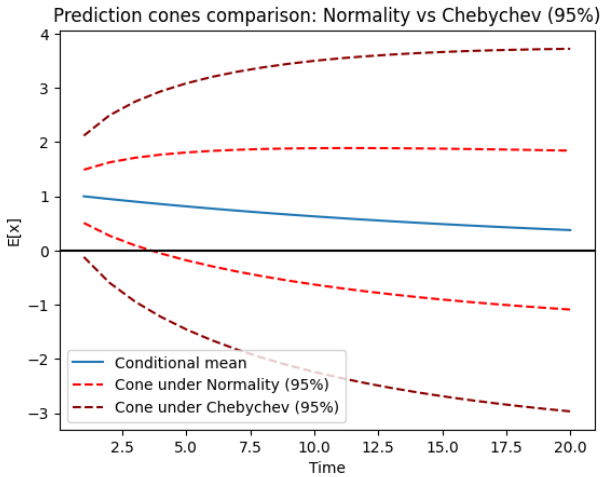


(c) 90% and 95% comparison

Comparison between Chebyshev inequality and normality assumption



(a) Normality vs Chebyshev (90%)



(b) Normality vs Chebyshev (95%)

Figure 6: Chebyshev versus normality

The Chebyshev bands are much wider and therefore more conservative. As the horizon increases, both cones widen because forecast uncertainty accumulates over time. The center of both cones is the same: the conditional mean.

The Chebyshev prediction cones are substantially wider than the Gaussian ones. This is expected

because Chebyshev's inequality is distribution-free and therefore conservative, while the normal bands rely on a stronger assumption and are more informative when normality is plausible.

Chebyshev provides a guarantee, but it is weak, since its bands are larger, they are more conservative. Normal bands give a more realistic interval if the shocks are close to Gaussian. The trade-off is robustness versus tightness.

iv) AR(1) paths simulation

To complement the theoretical prediction cones, 20 AR(1) paths were simulated starting from the same initial shock. This allows to visualize how individual realizations evolve around the conditional mean and how quickly they revert toward the long-run level. We first plot the simulated paths alone, then overlay them with the 90% prediction cone, and finally with the 95% prediction cone.

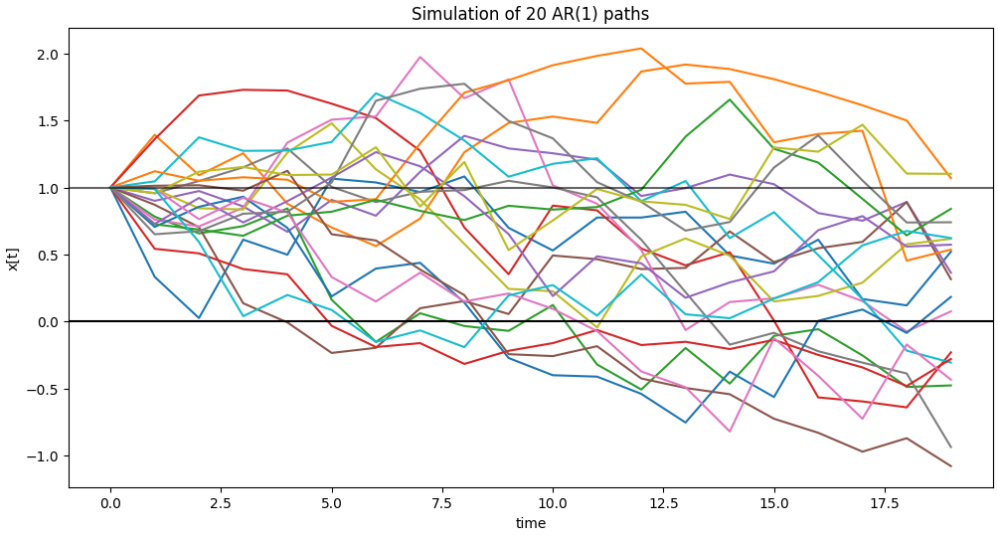


Figure 7: Simulation of 20 AR(1) paths

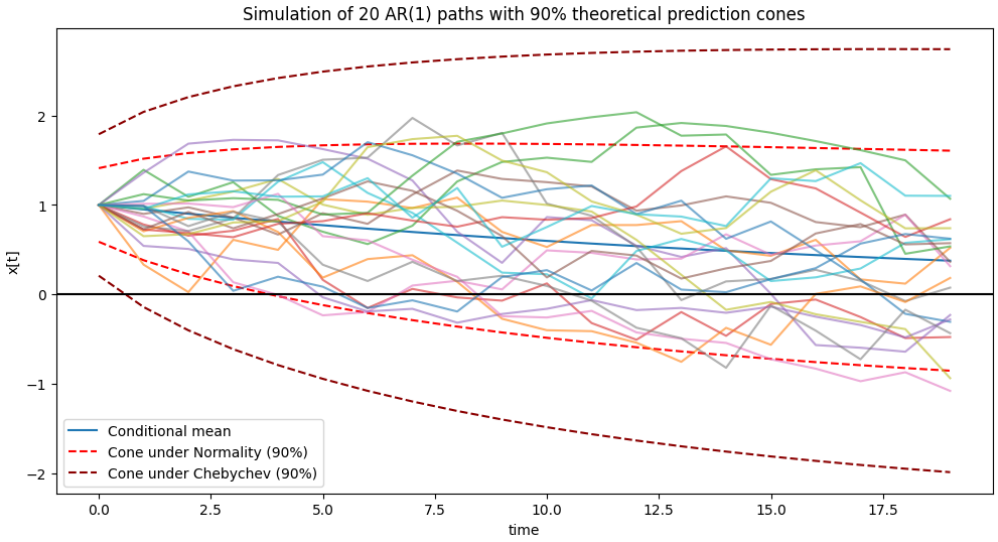


Figure 8: 20 paths with 90% prediction cones

Figure 8 overlays the simulated paths with the 90% prediction cone. It shows that several paths cross the Gaussian bands, especially at a few extreme dates, while most trajectories remain inside the wider Chebyshev bands. This is consistent with the fact that the normal bands are tighter and therefore less conservative.

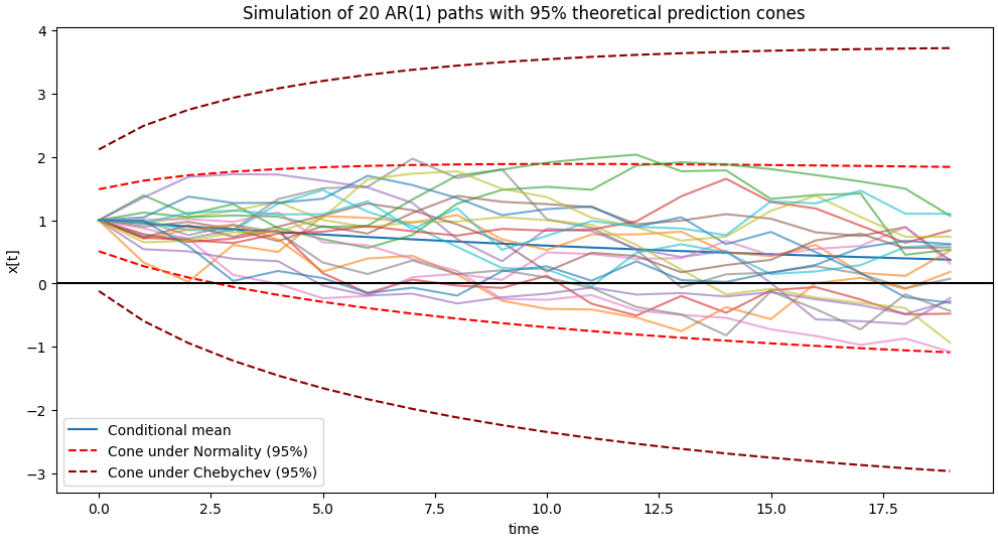


Figure 9: 20 paths with 95% prediction cones

Figure 9 exhibits the simulated paths with the 95% prediction cone. At this confidence level, the normal bands are still tighter than the Chebyshev bands, but they cover the simulated trajectories more comfortably. Even so, some tail shocks remain outside the normal interval, confirming that the Gaussian approximation understates extreme moves in a few simulations.

Overall, the figures confirm the interpretation from part (iii): the normality-based cones are more informative but rely on a stronger assumption, whereas the Chebyshev cones are distribution-free and therefore much more conservative. In both cases, the simulated paths illustrate mean reversion toward the long-run mean and the gradual increase in uncertainty with the horizon.

d) Unconditional Mean & Variance

The unconditional mean is defined as the limit of the conditional mean as the forecast horizon goes to infinity:

$$E(x_t) = \lim_{j\to\infty} E_t(x_{t+j}) = \lim_{j\to\infty} \rho^j x_t.$$

Since $|\rho| < 1$, we have $\rho^j \to 0$, so

$$\lim_{j\to\infty} \rho^j x_t = \lim_{j\to\infty} 0 = 0.$$

For the unconditional variance, we use the same idea and let the forecast horizon tend to infinity:

$$\mathrm{Var}(x_t) = \lim_{j\to\infty} \mathrm{Var}_t(x_{t+j}) = \lim_{j\to\infty} \sigma_\varepsilon^2 \frac{1-\rho^{2j}}{1-\rho^2}.$$

Since $|\rho| < 1$, we have $\rho^{2j} \to 0$, so

$$\lim_{j\to\infty} \sigma_\varepsilon^2 \frac{1-0}{1-\rho^2} = \frac{\sigma_\varepsilon^2}{1-\rho^2}.$$

Therefore, the unconditional moments are

$$\boxed{E(x_t) = 0, \qquad \mathrm{Var}(x_t) = \frac{\sigma_\varepsilon^2}{1-\rho^2}.}$$

1.2 Mean and variance of long-horizon returns

a) Annualized mean of long horizon returns

We know that $E(r_t) = \mu$.

So,

$$E(r_{t+1} + r_{t+2} + \cdots + r_{t+k}) = E\left(\sum_{i=1}^{k} r_{t+i}\right) = \sum_{i=1}^{k} E(r_{t+i}) = \sum_{i=1}^{k} \mu = k\mu.$$

Now,

$$E\left(\frac{1}{k}(r_{t+1} + r_{t+2} + \cdots + r_{t+k})\right) = E\left(\frac{1}{k}\sum_{i=1}^{k} r_{t+i}\right) = \frac{1}{k}E\left(\sum_{i=1}^{k} r_{t+i}\right)$$

$$= \frac{1}{k}\sum_{i=1}^{k} E(r_{t+i}) = \frac{1}{k}\sum_{i=1}^{k} \mu = \frac{1}{k} \cdot k\mu = \mu.$$

If the yearly mean is 6%, then

$$\boxed{textmonthlymean = \frac{6\%}{12} = 0.5\%.}$$

And the annualized monthly mean is

$$\boxed{0.5\% \times 12 = 6\%.}$$

b) Variance of long-horizon returns

We know that

$$\mathrm{Var}(r_t) = \sigma^2.$$

And, since the returns are not correlated over time,

$$\mathrm{Cov}(r_{t+i}, r_{t+j}) = 0 \quad \text{for } i \neq j.$$

So,

$$\mathrm{Var}(r_{t+1} + r_{t+2} + \cdots + r_{t+k}) = \mathrm{Var}\left(\sum_{i=1}^{k} r_{t+i}\right) = \mathrm{Var}\left(\sum_{i=1}^{k} r_{t+i}\right)$$

$$= \sum_{i=1}^{k} \mathrm{Var}(r_{t+i}) + 2\sum_{i<j} \mathrm{Cov}(r_{t+i}, r_{t+j}) = \sum_{i=1}^{k} \mathrm{Var}(r_{t+i}) = \sum_{i=1}^{k} \sigma^2 = k\sigma^2.$$

Now,

$$\mathrm{Var}\left(\frac{1}{k}(r_{t+1} + r_{t+2} + \cdots + r_{t+k})\right) = \mathrm{Var}\left(\frac{1}{k}\sum_{i=1}^{k} r_{t+i}\right).$$

Using $\mathrm{Var}(aX) = a^2 \mathrm{Var}(X)$,

$$= \frac{1}{k^2} \mathrm{Var}\left(\sum_{i=1}^{k} r_{t+i}\right) = \frac{1}{k^2} \cdot k\sigma^2 = \frac{\sigma^2}{k}.$$

c) Standard deviation of long-horizon returns

From part (b), we have

$$\mathrm{Var}\left(\sum_{i=1}^{k} r_{t+i}\right) = k\sigma^2.$$

Therefore, the standard deviation of the long-horizon return is

$$\sqrt{\mathrm{Var}\left(\sum_{i=1}^{k} r_{t+i}\right)} = \sqrt{k\sigma^2} = \sqrt{k}\,\sigma.$$

For the annualized long-horizon return,

$$\mathrm{Var}\left(\frac{1}{k}\sum_{i=1}^{k} r_{t+i}\right) = \frac{\sigma^2}{k}.$$

Therefore, the standard deviation of the annualized long-horizon return is

$$\sqrt{\mathrm{Var}\left(\frac{1}{k}\sum_{i=1}^{k} r_{t+i}\right)} = \sqrt{\frac{\sigma^2}{k}} = \frac{\sigma}{\sqrt{k}}.$$

If the standard deviation of annual returns is 16%, then

$$\boxed{\sigma_m = \frac{16\%}{\sqrt{12}} \approx 4.62\%.}$$

For daily returns,

$$\boxed{\sigma_d = \frac{16\%}{\sqrt{252}} \approx 1.01\%.}$$

d) Annualizing a monthly mean and standard deviation

If the monthly mean is $\mu_m = 1\%$ and the monthly standard deviation is $\sigma_m = 1\%$.

Then the annualized mean is

$$\boxed{\mu_y = 12\mu_m = 12 \times 1\% = 12\%}$$

The annualized standard deviation is

$$\boxed{\sigma_y = \sqrt{12}\,\sigma_m = \sqrt{12} \times 1\% \approx 3.46\%}$$

e) Sharpe ratio

The Sharpe ratio of the k-period log return is

$$\frac{E\left(\sum_{i=1}^{k} r_{t+i}\right)}{\sqrt{\text{Var}\left(\sum_{i=1}^{k} r_{t+i}\right)}} = \frac{k\mu}{\sqrt{k}\sigma} = \sqrt{k}\,\frac{\mu}{\sigma}.$$

For the annualized return,

$$\frac{E\left(\frac{1}{k}\sum_{i=1}^{k} r_{t+i}\right)}{\sqrt{\text{Var}\left(\frac{1}{k}\sum_{i=1}^{k} r_{t+i}\right)}} = \frac{\mu}{\sigma/\sqrt{k}} = \sqrt{k}\,\frac{\mu}{\sigma}.$$

Thus, the Sharpe ratio depends on the horizon k: it increases with the length of the horizon.

f) How does the advice scale with horizon?

We know that

$$\mu = 8\%, \qquad \sigma = 20\%, \qquad \gamma = 2.$$

Hence,

$$\sigma^2 = (20\%)^2 = 4\%.$$

The stock share is

$$w^* = \frac{1}{\gamma}\frac{\mu}{\sigma^2} = \frac{1}{2}\cdot\frac{8\%}{4\%} = 1$$

So the investor should put all wealth in stocks.

Now consider the 10-year horizon. Under the assumption of independent returns,

$$\mu_{10} = 10\mu = 10 \times 8\% = 80\%,$$

and

$$\sigma_{10} = \sqrt{10}\sigma = \sqrt{10} \times 20\% \approx 63.2\%.$$

Therefore,

$$VAR_{10} = \sigma_{10}^2 \approx (63.2\%)^2 = 40\%.$$

The optimal stock share at the 10-year horizon is

$$\boxed{w_{10}^* = \frac{1}{\gamma}\frac{\mu_{10}}{\sigma_{10}^2} = \frac{1}{2}\cdot\frac{80\%}{40\%} = 1}$$

Thus, the optimal allocation is the same at 1 year and 10 years, meaning that a 10-year investor should not hold more stocks than a 1-year investor solely because the horizon is longer. Therefore,

under i.i.d. returns, the optimal equity allocation is horizon-invariant.

g) Fallacy of time-diversification

The client's mistake is to confuse the volatility of the annualized average return with the risk that matters for portfolio choice.

The formula given in the question is

$$\text{stock share} = \frac{1}{\text{risk aversion}} \frac{E(r)}{\text{Var}(r)}.$$

As shown in parts **1.2.a** and **1.2.b**, under the assumption that returns are independent over time, the k-period total return is

$$R_k = r_{t+1} + r_{t+2} + \cdots + r_{t+k}.$$

Then

$$E(R_k) = E(r_{t+1}) + E(r_{t+2}) + \cdots + E(r_{t+k}) = k\mu,$$

and

$$\text{Var}(R_k) = \text{Var}(r_{t+1} + r_{t+2} + \cdots + r_{t+k}) = k\sigma^2.$$

So the optimal stock share is

$$\boxed{w_k = \frac{1}{\gamma} \frac{E(R_k)}{\text{Var}(R_k)} = \frac{1}{\gamma} \frac{k\mu}{k\sigma^2} = \frac{1}{\gamma} \frac{\mu}{\sigma^2}}$$

Thus, the optimal allocation is independent of the investment horizon. A long-horizon investor should therefore not allocate more to stocks simply because returns are "safer in the long run."

The fallacy comes from looking at the volatility of the annualized average return, which indeed declines with $1/\sqrt{k}$, instead of looking at the risk of the total return relevant for portfolio choice.

h) Long-horizon returns under an MA(1) process

i.A) Properties of the MA(1) process

We have

$$r_t = \mu + \varepsilon_t + \delta\varepsilon_{t-1}, \qquad E(\varepsilon_t) = 0, \qquad \text{Var}(\varepsilon_t) = \sigma_\varepsilon^2,$$

and the shocks are independent over time:

We first compute the unconditional mean of r_t.

$$
\begin{aligned}
E(r_t) &= E(\mu + \varepsilon_t + \delta\varepsilon_{t-1}) \\
&= E(\mu) + \underbrace{E(\varepsilon_t)}_{=0} + \delta\,\underbrace{E(\varepsilon_{t-1})}_{=0} \\
E(r_t) &= \mu
\end{aligned}
$$

Next, we compute the unconditional variance of r_t:

$$
\begin{aligned}
\mathrm{Var}(r_t) &= \mathrm{Var}(\mu + \varepsilon_t + \delta\varepsilon_{t-1}) \\
&= E(r_t^2) - E(r_t)^2 \\
&= E\big[(\mu + \varepsilon_t + \delta\varepsilon_{t-1})^2\big] - \mu^2 \\
&= E\big[\mu^2 + \varepsilon_t^2 + \delta^2\varepsilon_{t-1}^2 + 2\delta\varepsilon_t\varepsilon_{t-1} + 2\mu(\varepsilon_t + \delta\varepsilon_{t-1})\big] - \mu^2.
\end{aligned}
$$

As we know:

$$
E(\varepsilon_t) = 0, \qquad E(\varepsilon_{t-1}) = 0, \qquad E(\varepsilon_t\varepsilon_{t-1}) = 0, \qquad E(\varepsilon_t^2) = \sigma_\varepsilon^2, \qquad E(\varepsilon_{t-1}^2) = \sigma_\varepsilon^2.
$$

The expression becomes:

$$
\begin{aligned}
\mathrm{Var}(r_t) &= \mu^2 + \sigma_\varepsilon^2 + \delta^2\sigma_\varepsilon^2 + 0 + 0 - \mu^2 \\
&= (1 + \delta^2)\sigma_\varepsilon^2.
\end{aligned}
$$

We now turn to the covariance between r_t and r_{t+1}:

$$
\begin{aligned}
\mathrm{Cov}(r_t, r_{t+1}) &= E(r_t r_{t+1}) - E(r_t)E(r_{t+1}) \\
&= E\big[(\mu + \varepsilon_t + \delta\varepsilon_{t-1})(\mu + \varepsilon_{t+1} + \delta\varepsilon_t)\big] - \mu^2 \\
&= E\big[\mu^2 + \mu\varepsilon_{t+1} + \mu\delta\varepsilon_t + \mu\varepsilon_t + \mu\varepsilon_{t+1} + \mu\delta\varepsilon_t \\
&\quad + \varepsilon_t\varepsilon_{t+1} + \delta\varepsilon_{t-1}\varepsilon_{t+1} + \delta\varepsilon_t^2 + \delta\varepsilon_{t-1}\varepsilon_t \\
&\quad + \delta\varepsilon_{t-1}\varepsilon_{t+1} + \delta^2\varepsilon_{t-1}\varepsilon_t\big] - \mu^2 \\
&= \mu^2 + \mu\,\underbrace{E(\varepsilon_{t+1})}_{=0} + \mu\delta\,\underbrace{E(\varepsilon_t)}_{=0} + \mu\,\underbrace{E(\varepsilon_t)}_{=0} + \mu\,\underbrace{E(\varepsilon_{t+1})}_{=0} + \mu\delta\,\underbrace{E(\varepsilon_t)}_{=0} \\
&\quad + \underbrace{E(\varepsilon_t\varepsilon_{t+1})}_{=0} + \delta\,\underbrace{E(\varepsilon_{t-1}\varepsilon_{t+1})}_{=0} + \delta\,\underbrace{E(\varepsilon_t^2)}_{=\sigma_\varepsilon^2} + \delta\,\underbrace{E(\varepsilon_{t-1}\varepsilon_t)}_{=0} \\
&\quad + \delta\,\underbrace{E(\varepsilon_{t-1}\varepsilon_{t+1})}_{=0} + \delta^2\,\underbrace{E(\varepsilon_{t-1}\varepsilon_t)}_{=0} - \mu^2 \\
&= \mu^2 + \delta\sigma_\varepsilon^2 - \mu^2 \\
&= \delta\sigma_\varepsilon^2
\end{aligned}
$$

Finally, we compute the covariance between r_t and r_{t+2}:

$$\begin{aligned}
\mathrm{Cov}(r_t, r_{t+2}) &= E(r_t r_{t+2}) - E(r_t)E(r_{t+2}) \\
&= E\big[(\mu + \varepsilon_t + \delta\varepsilon_{t-1})(\mu + \varepsilon_{t+2} + \delta\varepsilon_{t+1})\big] - \mu^2 \\
&= E\big[\mu^2 + \mu\varepsilon_{t+2} + \mu\delta\varepsilon_{t+1} + \mu\varepsilon_t + \mu\varepsilon_{t+2} + \mu\delta\varepsilon_{t+1} \\
&\quad + \varepsilon_t\varepsilon_{t+2} + \delta\varepsilon_{t-1}\varepsilon_{t+2} + \delta\varepsilon_t\varepsilon_{t+1} + \delta\varepsilon_{t-1}\varepsilon_{t+1} \\
&\quad + \delta^2\varepsilon_{t-1}\varepsilon_{t+1}\big] - \mu^2 \\
&= \mu^2 + \mu\underbrace{E(\varepsilon_{t+2})}_{=0} + \mu\delta\underbrace{E(\varepsilon_{t+1})}_{=0} + \mu\underbrace{E(\varepsilon_t)}_{=0} + \mu\underbrace{E(\varepsilon_{t+2})}_{=0} + \mu\delta\underbrace{E(\varepsilon_{t+1})}_{=0} \\
&\quad + \underbrace{E(\varepsilon_t\varepsilon_{t+2})}_{=0} + \delta\underbrace{E(\varepsilon_{t-1}\varepsilon_{t+2})}_{=0} + \delta\underbrace{E(\varepsilon_t\varepsilon_{t+1})}_{=0} + \delta\underbrace{E(\varepsilon_{t-1}\varepsilon_{t+1})}_{=0} \\
&\quad + \delta^2\underbrace{E(\varepsilon_{t-1}\varepsilon_{t+1})}_{=0} - \mu^2 \\
&= \mu^2 - \mu^2 \\
&= 0
\end{aligned}$$

This shows that the MA(1) process has finite memory: returns are correlated with the previous shock, so $\mathrm{Cov}(r_t, r_{t+1}) \neq 0$, but returns two periods apart are uncorrelated, so $\mathrm{Cov}(r_t, r_{t+2}) = 0$. Hence, the process captures the short-term only, with dependence disappearing after one lag.

i.B) Impulse-response functions for momentum and mean reversion

We consider the impulse response of returns and cumulative log returns to a one-time shock, $\varepsilon_1 = 1$, under three values of δ.

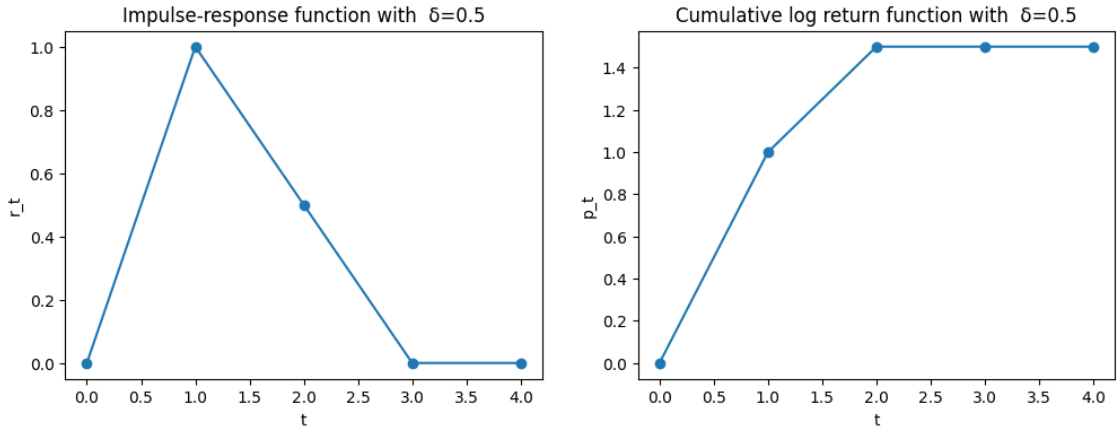


Figure 10: Momentum: $\delta = 0.5$

For $\delta = 0.5$, the shock persists through time: the return response remains positive after the initial shock, which reflects momentum. The cumulative log return therefore, increases more than under the i.i.d. benchmark and stays at a higher level.

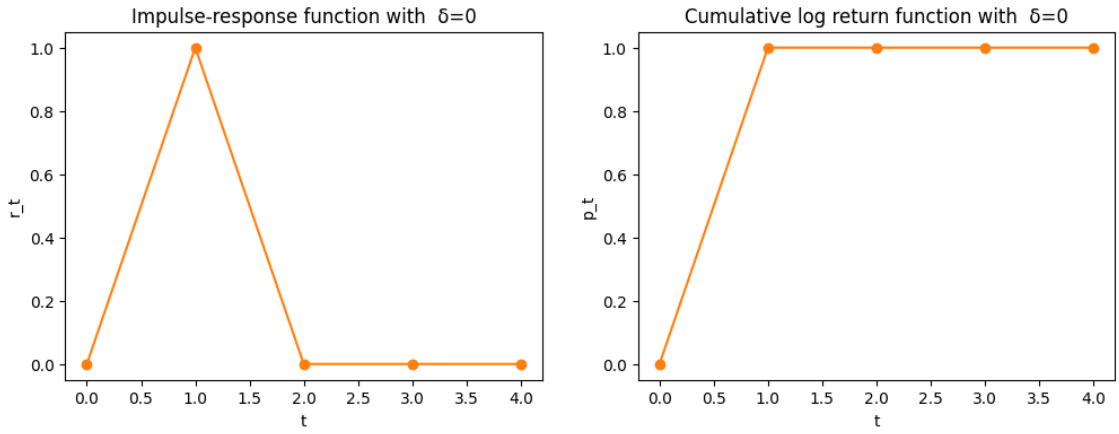


Figure 11: i.i.d. benchmark: $\delta = 0$

For $\delta = 0$, the shock has a one-time effect on returns and no further dynamic impact. This is the i.i.d. benchmark: returns are not persistent, while the cumulative log return is permanently shifted by one unit.

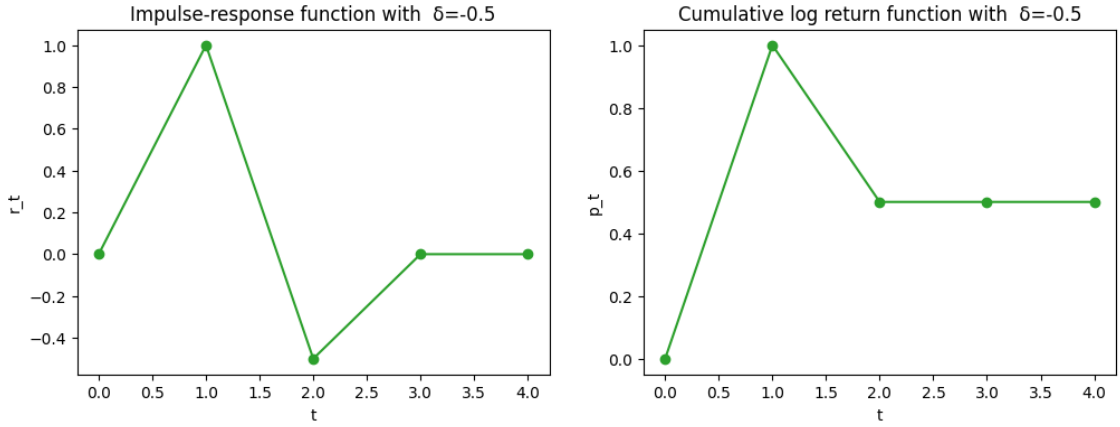


Figure 12: Mean reversion: $\delta = -0.5$

For $\delta = -0.5$, the initial positive shock is partly reversed in the next period. This reflects mean reversion: the return response becomes negative after the shock, and the cumulative log return settles at a lower permanent level than in the momentum case.

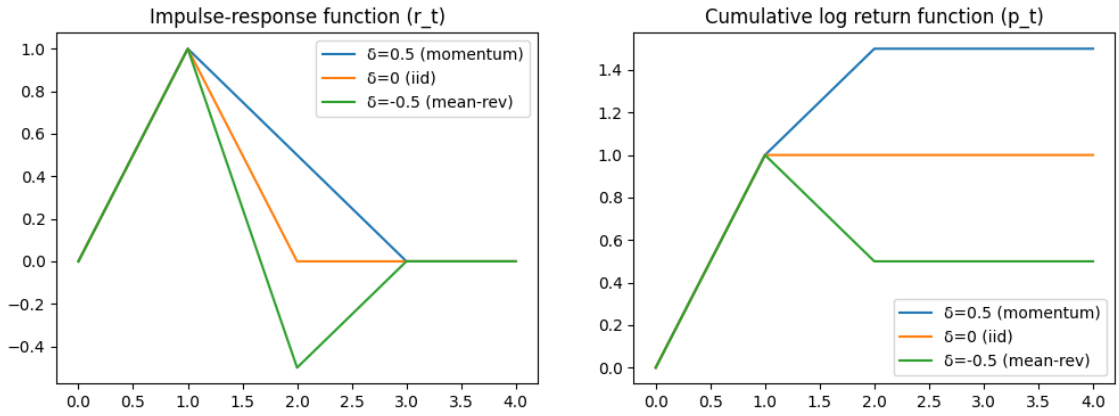


Figure 13: Comparison of impulse responses

Overall, the comparison shows that momentum amplifies and prolongs the effect of shocks, whereas mean reversion dampens them and partially reverses the initial movement.

ii) Long-horizon mean and variance

We know that

$$E(r_t) = \mu, \qquad \mathrm{Var}(\varepsilon_t) = \sigma_\varepsilon^2, \qquad \mathrm{Cov}(r_t, r_{t+1}) = \delta\sigma_\varepsilon^2, \qquad \mathrm{Cov}(r_t, r_{t+2}) = 0.$$

First, the mean long-horizon return is

$$\begin{aligned} E\left(\sum_{i=1}^{k} r_{t+i}\right) &= \sum_{i=1}^{k} E(r_{t+i}) \\ &= \sum_{i=1}^{k} \mu \\ &= k\mu. \end{aligned}$$

So δ does not affect the mean.
Next, the variance of the long-horizon return is

$$\begin{aligned} \mathrm{Var}\left(\sum_{i=1}^{k} r_{t+i}\right) &= \sum_{i=1}^{k} \mathrm{Var}(r_{t+i}) + 2\sum_{i<j} \mathrm{Cov}(r_{t+i}, r_{t+j}) \\ &= k\,\mathrm{Var}(r_t) + 2(k-1)\,\mathrm{Cov}(r_t, r_{t+1}) \\ &= k(1+\delta^2)\sigma_\varepsilon^2 + 2(k-1)\delta\sigma_\varepsilon^2 \\ &= \sigma_\varepsilon^2\big(k(1+\delta^2) + 2(k-1)\delta\big). \end{aligned}$$

For a null long-horizon variance $\delta = 0$, this collapses to the i.i.d. benchmark $k\sigma_\varepsilon^2$. If $\delta > 0$ is larger, so momentum makes long-horizon returns riskier. If $\delta < 0$, long-horizon variance is smaller than under i.i.d., so mean reversion makes long-horizon returns safer.

2 Stock Market

2.1 Efficient Frontier

Part 1 - Two risky assets and one risk-free asset

1. Data Collection and Frequency Conversion

Daily adjusted closing prices for the Vanguard European Stock Index Fund (VEUSX) and the Vanguard Long-Term Bond Index Fund (VBLIX) were retrieved from Yahoo Finance over the period January 1, 2015 to today, using the yfinance Python library. Prices were downloaded at daily frequency and subsequently `converted to monthly frequency by retaining the last available closing price of each calendar month`. This resampling approach ensures that each monthly observation corresponds to an actual traded price, thereby avoiding any interpolation bias.

The use of split- and dividend-adjusted closing prices (`auto_adjust=True`) ensures that the series accurately reflects the total return performance of each fund, accounting for dividend distributions and capital adjustments over time. Monthly simple returns were then computed as:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

yielding a final sample of 135 monthly return observations spanning February 2015 to April 2026. These return series form the basis for all subsequent portfolio computations, including the estimation of the sample mean vector $\boldsymbol{\mu}$ and the variance-covariance matrix $\boldsymbol{\Sigma}$.

2. Normalized prices chart

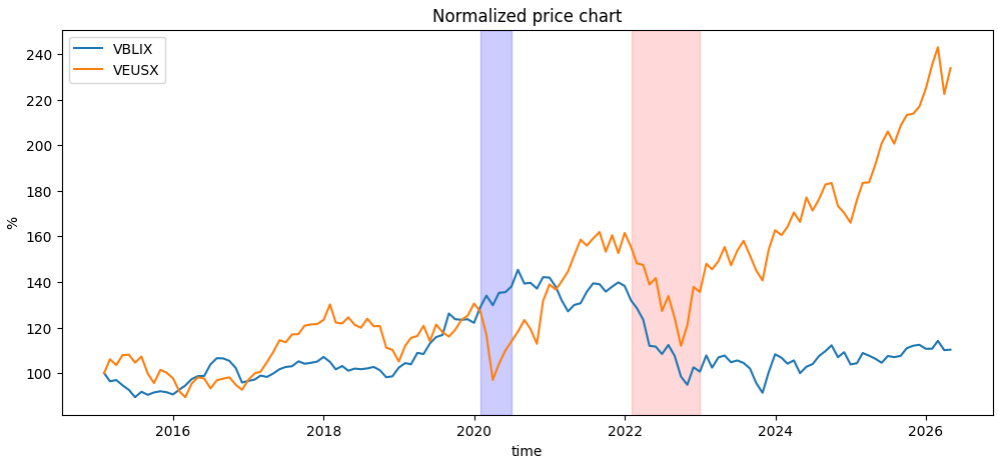


Figure 14: Normalized price chart

This chart displays the normalized price series of both funds, rebased to 100 in January 2015. Two major macroeconomic shocks are highlighted:

1) The COVID-19 pandemic (February–June 2020, shaded in blue) triggered a sharp decline in the Vanguard European Stock Index Fund (VEUSX), reflecting the broad equity sell-off observed

in global markets during the onset of the crisis. The Vanguard Long-Term Bond Fund (VBLIX), by contrast, exhibited a brief upward spike followed by a rapid reversion, consistent with a short-lived flight-to-safety dynamic into sovereign fixed income. This episode illustrates a non-correlation between equities and long-duration bonds during periods of risk.

2) The outbreak of the war in Ukraine (February–December 2022, shaded in red) produced a markedly different pattern. Both VEUSX and VBLIX declined simultaneously, breaking the negative correlation assumptions. This co-movement was driven by the inflationary pressures, which prompted monetary tightening by major central banks. Rising interest rates simultaneously compressed equity valuations and eroded the market value of long-duration bonds.

3. Sample moments

The sample mean is computed as:

$$\hat{\mu}_i = \frac{1}{T}\sum_{t=1}^{T} r_{i,t}$$

Which gives us:

Ticker	Mean Monthly Return
VBLIX	0.1242 %
VEUSX	0.7476 %

Table 1: Sample Mean Vector (monthly returns)

and the sample covariance between assets i and j as:

$$\hat{\sigma}_{ij} = \frac{1}{T-1}\sum_{t=1}^{T}(r_{i,t} - \hat{\mu}_i)(r_{j,t} - \hat{\mu}_j)$$

Which gives us:

	VBLIX	VEUSX
VBLIX	0.001045	0.000662
VEUSX	0.000662	0.002337

Table 2: Sample Variance-Covariance Matrix (monthly returns)

VEUSX exhibits a higher mean monthly return (0.748%) relative to VBLIX (0.124%), accompanied by a higher variance, consistent with the equity risk premium. The positive off-diagonal element of $\mathbf{\Sigma}$ ($\hat{\sigma}_{12} = 0.000662$) implies a positive sample correlation ($\hat{\rho} = 0.4238$) between the two funds.

4. Efficient frontier

The efficient frontier is constructed by systematically varying the portfolio weight w allocated to VBLIX, with the complementary weight $(1 - w)$ assigned to VEUSX, such that the full investment constraint $w + (1 - w) = 1$ is satisfied at all times. For each weight w, the portfolio expected return and standard deviation are computed as:

$$\mu_p = \mathbf{w}^\top \boldsymbol{\mu}$$
$$\sigma_p = \sqrt{\mathbf{w}^\top \boldsymbol{\Sigma}\, \mathbf{w}}$$

No short-selling constraint is imposed, in line with the standard mean-variance framework of Markowitz 1952. Accordingly, the weight w is swept over the interval $[-1, 1.5]$, allowing for leveraged long positions (weights exceeding 1) as well as short positions (negative weights). This ensures that the full hyperbola is traced in the mean–standard deviation plane, rather than a constrained segment.

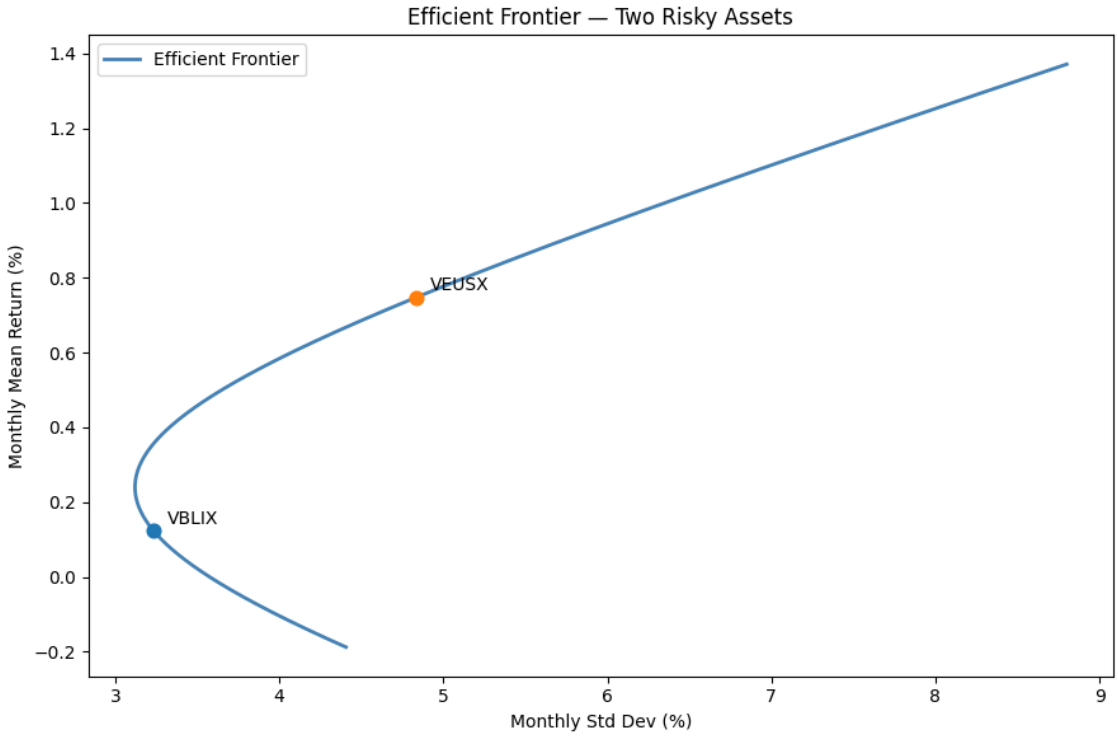


Figure 15: Efficient frontier drawn by VBLIX and VEUSX

The frontier represents the set of portfolios that achieve the highest expected return for a given level of risk, or equivalently, the lowest variance for a given level of expected return.

Any portfolio lying below the frontier is sub-optimal. Rational, risk-averse investors will therefore only hold portfolios located on the upper segment of the frontier.

5. Minimum Variance Portfolio

The Global Minimum Variance Portfolio (GMVP) is obtained by minimizing the portfolio variance $\sigma_p^2 = \mathbf{w}^\top \boldsymbol{\Sigma}\, \mathbf{w}$ with respect to the weight w, subject to the full investment constraint $w + (1 - w) = 1$. In the two-asset case, this minimization yields the following closed-form solution:

$$w^*_{\text{VBLIX}} = \frac{\sigma^2_{\text{VEUSX}} - \sigma_{\text{VBLIX,VEUSX}}}{\sigma^2_{\text{VBLIX}} + \sigma^2_{\text{VEUSX}} - 2\sigma_{\text{VBLIX,VEUSX}}}$$

with the complementary weight $w^*_{\text{VEUSX}} = 1 - w^*_{\text{VBLIX}}$.

	Monthly	Annualized
Weight VBLIX	0.8141	—
Weight VEUSX	0.1859	—
Mean Return	0.24%	2.88%
Std Deviation	3.12%	10.80%
Sharpe Ratio	0.069	0.239

Table 3: Global Minimum Variance Portfolio — Performance Summary

The GMVP assigns a weight of 81.41% to the bond fund and 18.59% to stocks. This heavy allocation toward the bond fund is consistent with its lower individual variance, which dominates despite the positive sample covariance between the two assets. The portfolio achieves a monthly standard deviation of 3.12%, lower than either asset in isolation ($\sigma_{\text{VBLIX}} = 3.2327\%$, $\sigma_{\text{VEUSX}} = 4.8344\%$). However, with an annualized Sharpe ratio of 0.2388, the GMVP sacrifices return in exchange for variance minimization. Risk-adjusted performance is maximized at the tangency portfolio, as discussed in the following section.

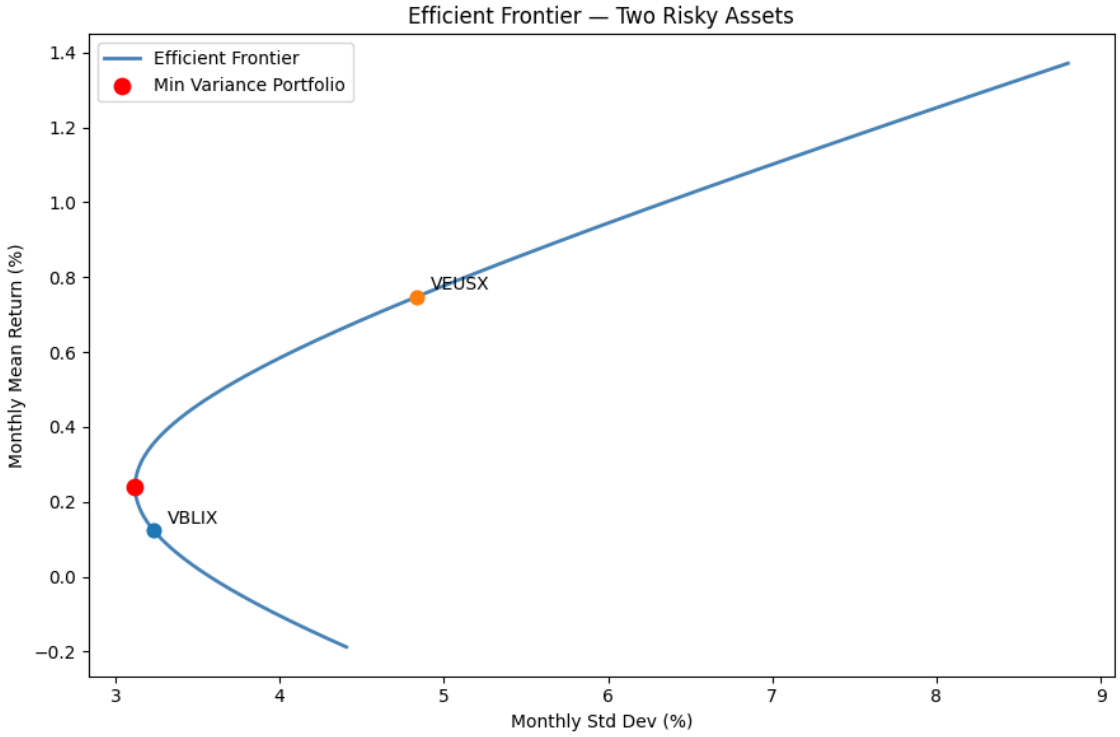


Figure 16: Efficient frontier with the GMVP

6. Tangency portfolio

The tangency portfolio is the risky portfolio that maximizes the Sharpe ratio, defined as the slope of the Capital Market Line (CML) connecting the risk-free asset to the efficient frontier. Given a monthly risk-free rate of $r_f = 0.025\%$, the tangency weights are obtained analytically as:

$$\mathbf{w}_T = \frac{\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r_f \mathbf{1})}{\mathbf{1}^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r_f \mathbf{1})}$$

where $(\boldsymbol{\mu} - r_f \mathbf{1})$ is the vector of excess returns over the risk-free rate. The normalization by $\mathbf{1}^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r_f \mathbf{1})$ ensures that the portfolio weights sum to one. Once again, short selling and leverage is allowed.

	Monthly	Annualized
Weight VBLIX	−0.5572	—
Weight VEUSX	1.5572	—
Mean Return	1.0949%	13.1384%
Std Deviation	6.9585%	24.1050%
Sharpe Ratio	0.1537	0.5326

Table 4: Tangency Portfolio — Performance Summary

The tangency portfolio assigns a weight of 155.72% to VEUSX and a weight of −55.72% to VBLIX, implying a leveraged long position in the equity fund and a short position in the bond fund.
This result is a direct consequence of the significant return differential between the two assets: VEUSX substantially outperforms VBLIX on a risk-adjusted basis, combined with their positive sample covariance, which reduces the diversification benefit of holding bonds. In the absence of short-selling constraints, the optimizer systematically shorts the lower-performing asset to amplify exposure to the higher Sharpe ratio asset.
The tangency portfolio achieves an annualized Sharpe ratio of 0.5326, materially higher than that of the GMVP (0.2388), confirming that variance minimization and Sharpe maximization are distinct objectives leading to different optimal allocations.

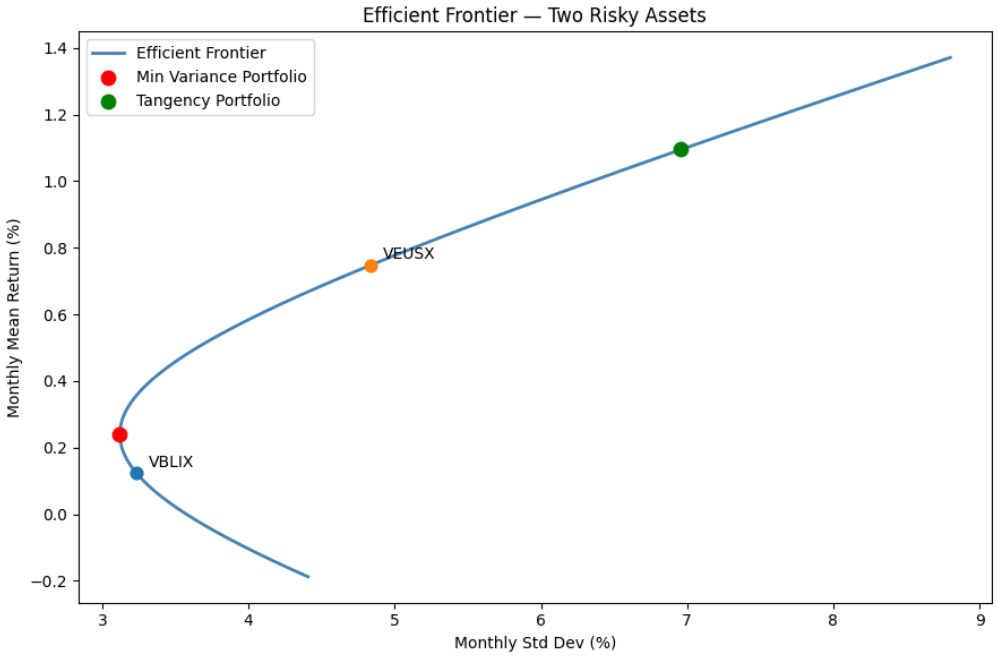


Figure 17: Efficient frontier with the GMVP & Tangency Portfolio

Part 2 - Five risky assets and one risk-free asset

This section extends the efficient frontier analysis to a five-asset universe comprising Google (GOOG), Cisco Systems (CSCO), Logitech International (LOGI), Amazon (AMZN), and Apple (AAPL), using monthly returns from January 2015.

1. Data Collection and Statistics

Before constructing the efficient frontier, we examine the individual characteristics of the five assets. These individual profiles suggest that diversification gains will be most significant for high-volatility assets such as LOGI and AMZN, and that the tangency portfolio might overweight GOOG as it exhibits the best sharpe ratio.

Asset	Mean (%)	Std (%)	Sharpe
GOOG	2.1934	7.4043	0.2929
CSCO	1.3540	6.5959	0.2015
LOGI	1.9540	8.9680	0.2151
AMZN	2.3886	8.9236	0.2649
AAPL	2.0463	7.7745	0.2600

Table 5: Descriptive Statistics of Monthly Returns (Jan 2015 – Present)

Let's look at the pairwise correlation matrix of monthly returns for the five assets. All correlations are positive, ranging from 0.33 to 0.63, reflecting the common exposure of these assets to broad US equity market risk.

Conversely, LOGI exhibits the lowest correlations across the board (0.33–0.39), suggesting it provides the greatest diversification benefit within the portfolio despite its high individual volatility.
Notably, no pair exhibits a negative correlation, which limits the scope for variance reduction through diversification alone and reinforces the role of the risk-free asset in constructing the optimal portfolio.

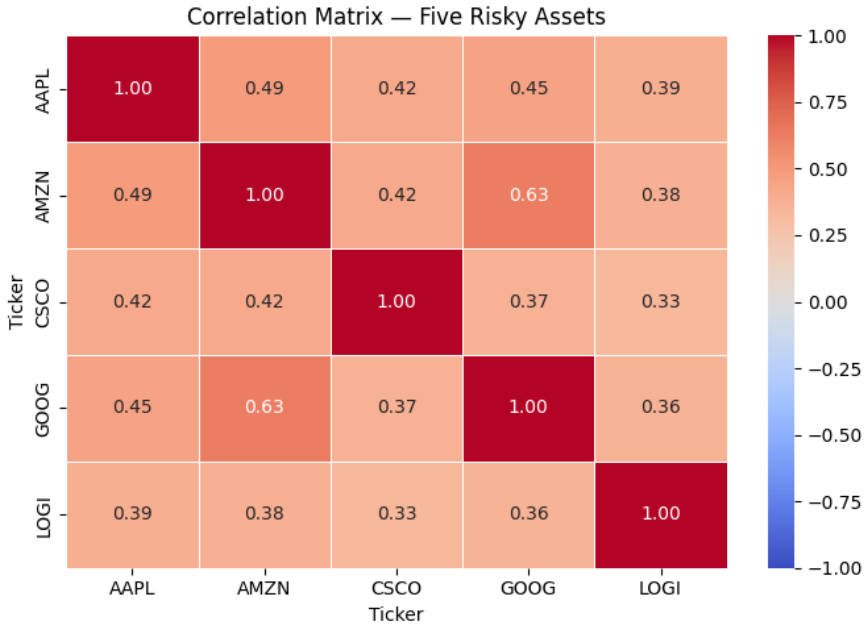


Figure 18: Correlation Matrix

2. Efficient frontier

The efficient frontier is constructed differently from part 1. As we have more than 2 assets, we have to solve for each target return μ^*, the following quadratic program:

$$\min_w \; w^\top \Sigma w \quad \text{s.t.} \quad w^\top \mathbf{1} = 1, \quad w^\top \mu = \mu^*$$

where $w \in R^5$ is the vector of portfolio weights, μ the vector of sample mean returns, and Σ the sample covariance matrix. The optimization is performed numerically via Sequential Least Squares Programming (SLSQP) over a grid of 300 target returns, allowing short-selling.

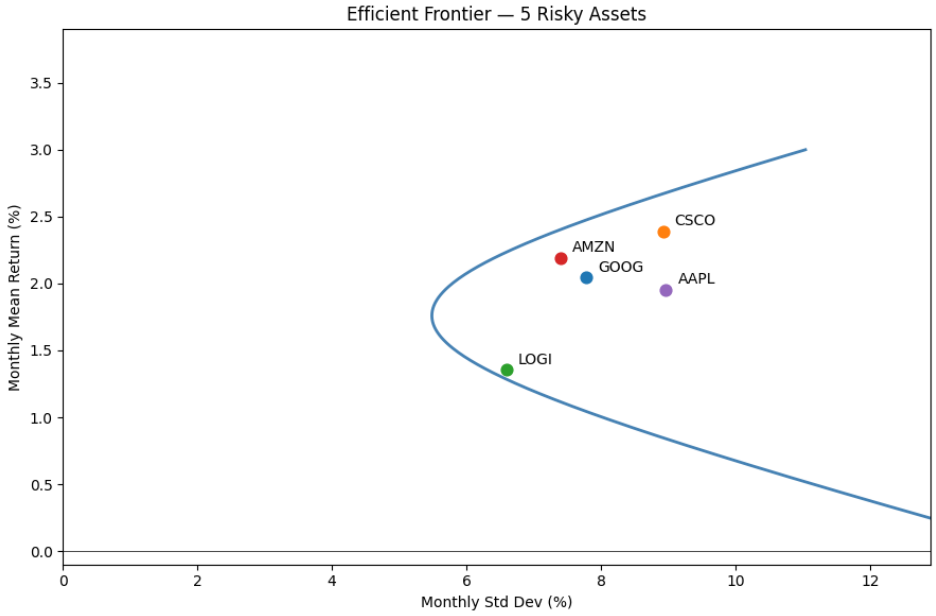


Figure 19: Efficient frontier

The efficient frontier is shifted to the right relative to Part 1, reflecting the high individual volatility of the five assets (monthly std ranging from 6.60% to 8.97%) and their positive pairwise correlations (0.33–0.63). Since all assets belong to the US technology and growth equity space, they share significant common risk exposure, limiting the variance reduction achievable through diversification.

3. Tangency & GMVP Portfolio

The tangency portfolio and the Global Minimum Variance Portfolio (GMVP) are derived using the same closed-form approach as in Part 1, applying the analytical solutions.

The tangency portfolio achieves a monthly Sharpe ratio of 0.3422, strictly dominating all individual assets on a risk-adjusted basis. AMZN and GOOG jointly account for 61% of the allocation, consistent with their superior individual Sharpe ratios.

The GMVP, with a monthly standard deviation of 5.48%, remains substantially more volatile than its Part 1 counterpart, reflecting the inherently higher risk of equity assets relative to a mixed stock-bond universe. To minimize variance, the optimizer heavily overweights LOGI (44.47%), exploiting its relatively low pairwise correlations with the remaining assets.

Table 6: Tangency Portfolio and GMVP — Key Statistics

	Tangency	**GMVP**
Mean (%)	2.0488	1.7519
Std (%)	5.9135	5.4827
Sharpe	0.3422	0.3150

Table 7: Tangency Portfolio and GMVP — Asset Weights

Asset	**Tangency (%)**	**GMVP (%)**
GOOG	24.16	20.83
CSCO	12.27	−2.47
LOGI	11.67	44.47
AMZN	37.25	23.46
AAPL	14.64	13.71
Total	**100.00**	**100.00**

4. Capital Market Line

For any portfolio standard deviation σ_p, the CML gives the maximum achievable expected return when combining the tangency portfolio with the risk-free asset:

$$\mu_p = r_f + \frac{\mu_{tan} - r_f}{\sigma_{tan}} \cdot \sigma_p$$

where $r_f = 0.025\%$ is the monthly risk-free rate, and $\frac{\mu_{tan} - r_f}{\sigma_{tan}} = 0.3422$ is the Sharpe ratio of the tangency portfolio. The CML is plotted over a grid of σ_p values ranging from 0 to $2\sigma_{tan}$.

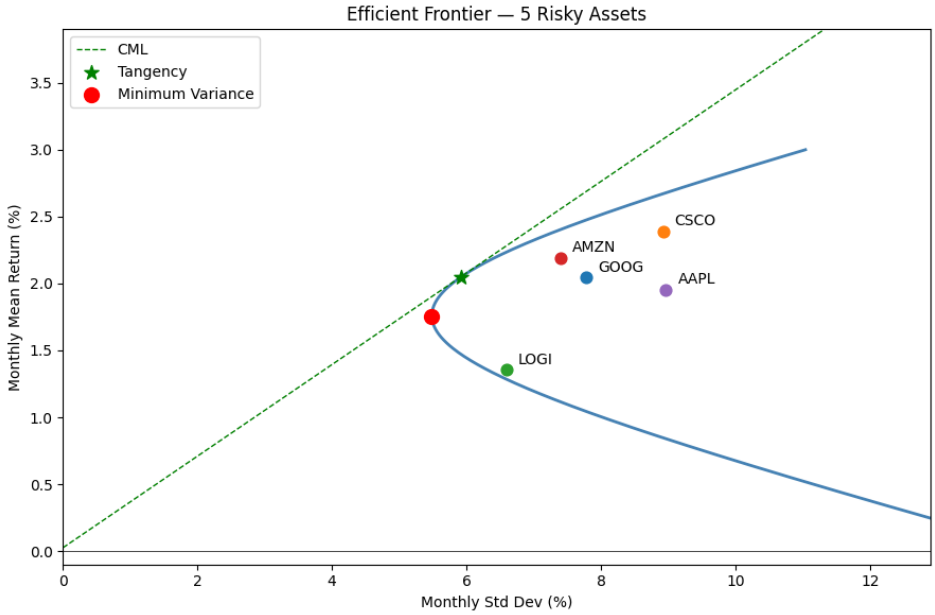


Figure 20: Efficient frontier & GMVP & Tangency portfolio & GML

The CML strictly dominates the risky-asset frontier for all levels of risk: any portfolio on the CML achieves a higher expected return than a portfolio on the frontier at the same standard deviation.

Furthermore, all individual assets lie below and to the right of the CML, implying that none of them is mean-variance efficient on its own.

Finally, the tangency portfolio, with a monthly Sharpe ratio of 0.3422, represents the unique point

of tangency between the CML and the efficient frontier, offering the highest risk-adjusted return achievable within this five-asset universe.

2.2 Stock returns and strategies

1) Rank-Based Industry Momentum

The industry momentum strategy relies on a cross-sectional ranking system. We process the 30 Industry Portfolios from 1926 to 2026.

1. **Data Preparation:** We compute the 12-month rolling average return for each industry, $\bar{R}_{i,t}$, using a rolling window of 12 months.

2. **Ranking:** Starting from 1927/07, for each month t, we rank industries based on their $\bar{R}_{i,t-1}$ (excluding the month itself) using:

$$r_{i,t} = \text{rank}(\bar{R}_{i,t-1}, \text{axis} = 1)$$

   where $r_{i,t} \in [1, 30]$. This ranks industries from the worst (1) to the best (30) performing over the previous year.

3. **Portfolio Construction:** We define the *Winner* portfolio as the top 15 industries ($r_{i,t} \geq 16$) and the *Loser* portfolio as the bottom 15 industries ($r_{i,t} \leq 15$).

Our analysis reveals a non-uniform distribution of historical performance across industries. The Telecom (*Telcm*) sector exhibits the lowest average rank (14.15), while the Business Equipment (*BusEq*) sector displays the highest (16.89).

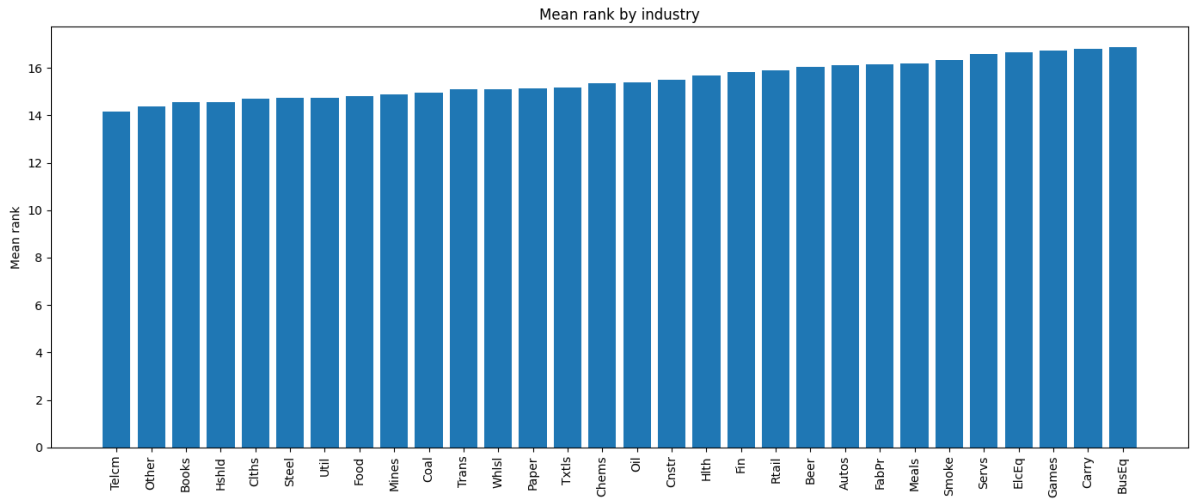


Figure 21: Mean rank by industry

The visualization of these mean ranks illustrates a relatively tight clustering around the median rank of 15.5, suggesting that while some momentum persistence exists, industries frequently oscillate in their relative performance standing.

The monthly rank autocorrelation equals 0.91, suggesting strong short-term persistence in industry performance. However, this high correlation is deceptive: it reflects the stability of the entire

cross-section rather than the stability of individual winners. The next figure illustrates this volatility using the Oil industry as an example, showing frequent swings between top and bottom rank deciles.

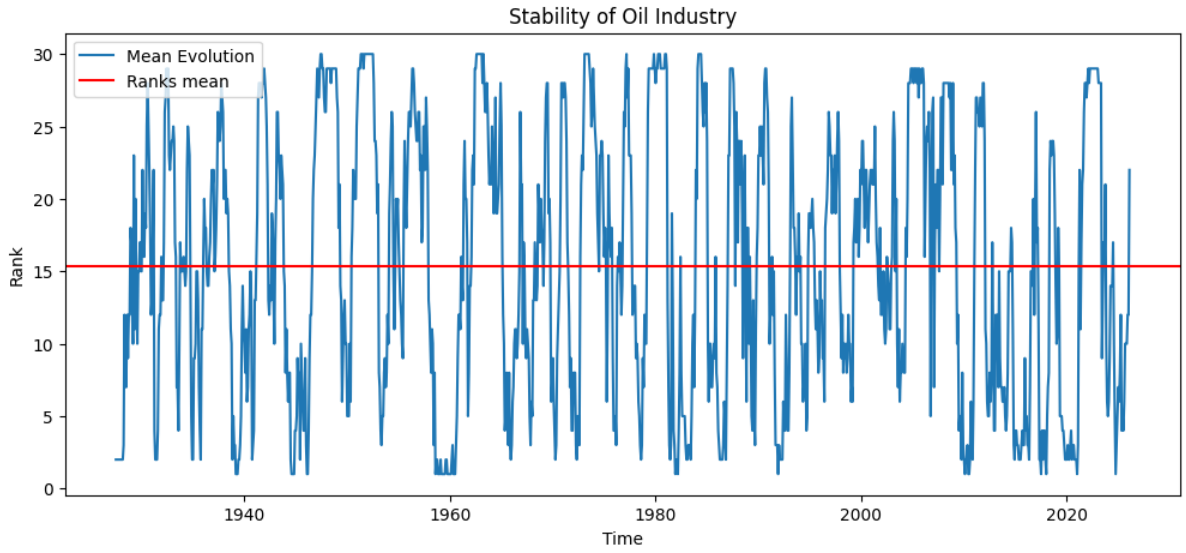


Figure 22: Oil industry rank evolution

To quantify the rebalancing intensity, we measure the turnover rate (ROT) of the top-5 industries from month to month:

$$ROT = \frac{1}{5}\sum_{i=1}^{30} I(rank_{i,t} \geq 26 \neq rank_{i,t-1} \geq 26)$$

where I is an indicator function tracking entries and exits from the top-tier. This quick analysis gives an average turnover of 44.89% for the top-5 industries, confirming that the composition of the portfolio shifts significantly month-to-month. Consequently, industry momentum is a highly dynamic strategy that demands frequent rebalancing.

2) Portfolio Performance Analysis

To construct the *Winner* and *Loser* portfolios, we assign equal weights to the industries ranking in the top and bottom 15, respectively. We compute monthly excess returns by subtracting the risk-free rate (RF), as provided by the Fama/French research data, from the total portfolio return. The market performance is benchmarked against the *Mkt-RF* factor, adjusted for the risk-free rate to reflect total market return.

Metric	**Winner**	**Loser**	**Market**
Monthly Excess Return (Mean)	0.81%	0.52%	0.65%
Monthly Std. Deviation	4.67%	6.00%	3.63%
Monthly Sharpe Ratio	0.1734	0.0861	0.1796
Annualized Sharpe Ratio	0.6006	0.2984	0.6220

Table 8: Summary of risk-adjusted performance of the three strategies

The *Winner* portfolio delivers a superior risk-adjusted return compared to the *Loser* portfolio, consistent with the momentum hypothesis. Notably, the *Winner* portfolio matches the market index in terms of Sharpe ratio (0.60 vs. 0.62) with substantially better excess returns.

The *Loser* portfolio exhibits the lowest excess returns, despite the highest volatility (6.00% vs. 4.67%) and the lowest Sharpe ratio, confirming that industries with poor past performance not only exhibit lower performances but also higher downside risk.

3) Performance plots

To evaluate long-term wealth accumulation, we compute the cumulative return for each strategy using the compound product of monthly returns: $CR_t = \prod_{i=1}^{t}(1 + R_i)$.
For the *Long/Short* industry momentum portfolio, we compute returns as the spread between the *Winner* and *Loser* portfolios ($R_{L/S} = R_W - R_L$). All cumulative returns are visualized on a logarithmic scale to better capture growth trajectories and comparative performance drawdowns.

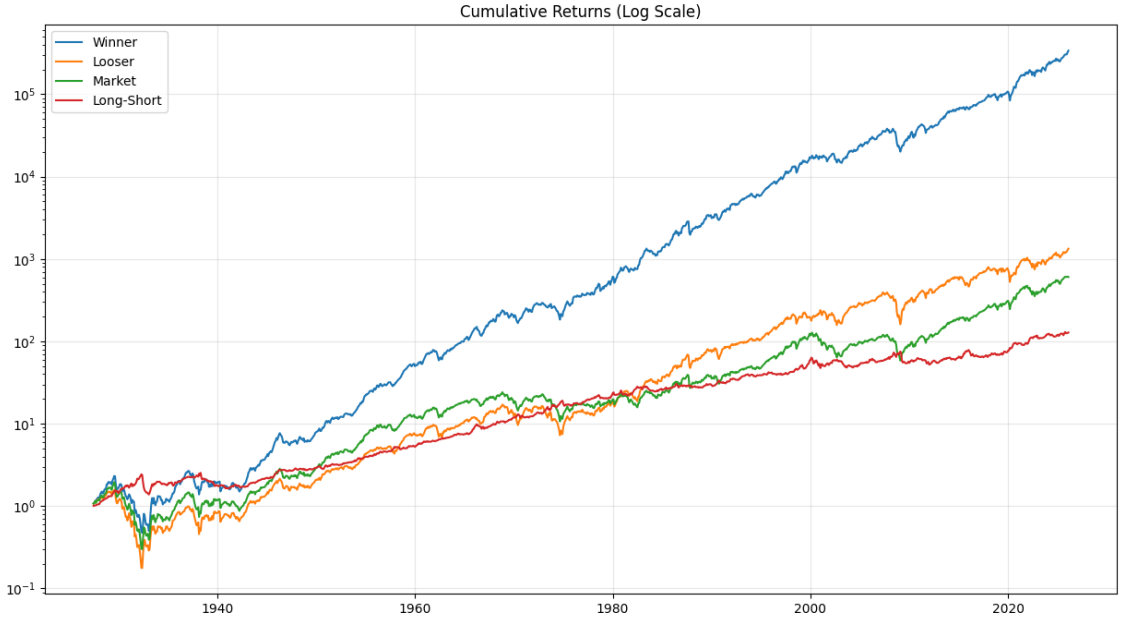


Figure 23: Cumulative returns plot

The *Winner portfolio* captures significant momentum, consistently outperforming the other portfolio and the market in terms of absolute cumulative returns.

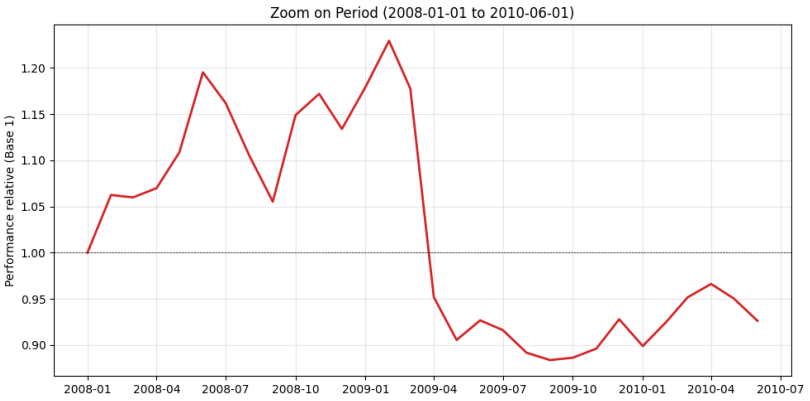
However, the raw *long/short industry momentum* strategy does not consistently beat the market on a risk-adjusted basis. These results demonstrate that naive industry momentum is not a dominant strategy; rather, it is a volatile dynamic bet that requires sophisticated filtering.

Furthermore, we observe that while the *Loser portfolio* exhibits a lower average monthly return than the market, it demonstrates competitive long-term cumulative growth. This phenomenon illustrates the interplay between volatility drag and mean reversion: high-beta industries captured in the Loser bucket during market recovery phases significantly inflate cumulative wealth, effectively masking the strategy's structural underperformance on a risk-adjusted basis.

4) Industry momentum losses

The industry momentum strategy is susceptible to "momentum crashes," typically occurring during sharp market reversals. In March, April, and May 2009, following the Global Financial Crisis, the long/short portfolio suffered a cumulative loss of approximately 28.2%, driven by a massive recovery in the *Loser* portfolio (+26.7% in April alone) while *Winners* underperformed the rapid market rebound.

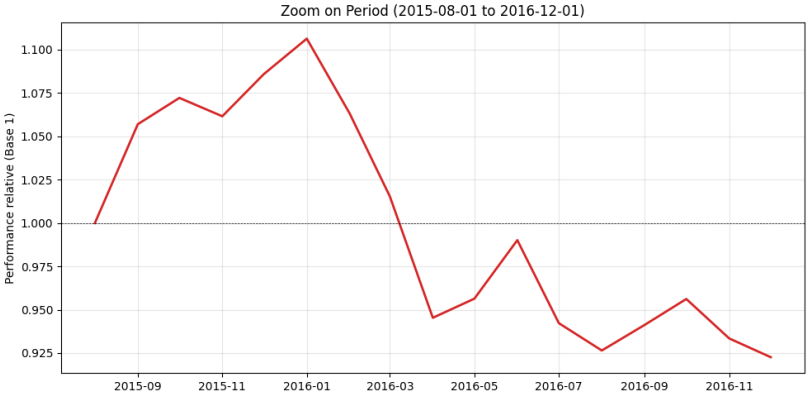
Portf.	Cum. Ret
Winner	19.38%
Loser	46.62%
L/S	-28.23%



It exemplifies the classic momentum trap. During market crises, *Loser* industries are often those with the highest market sensitivity (high-beta), so when the market rebounds, these high-beta industries experience a stronger rebound, outperforming the more defensive "winner" industries that had led the market down.

We identify a similar episode in early February, March, and April 2016, where the strategy lost 15.3% cumulatively. This crash was driven by a sharp reversal in commodity markets: throughout 2015, oil prices plummeted below 30$ per barrel, causing the energy-related sectors to become the worst performer and thus a key component of our *Loser* portfolio. When oil prices and energy stocks rebounded in early 2016, our strategy, which shorted these sectors, suffered from their recovery.

Portf.	Cum. Ret
Winner	5.76%
Loser	21.06%
L/S	-15.29%



These momentum crashes are systematic: they occur whenever market sentiment shifts abruptly from extreme pessimism to recovery, forcing a rapid and aggressive repricing of previously neglected, high-beta sectors.

3 Bond Market

3.1 Test 1: Long yield changes over the life of the short bond

a) Methodology

Campbell and Shiller (1991) study tests the expectations theory of the term structure, which says that the spread between a long rate and a short rate should predict future interest-rate movements. If the theory is correct, a high spread should imply that:

- the short rate will rise in the future;
- the long rate will move in a way consistent with expected future short rates.

To test this hypothesis, we first estimate the relation between the future change in the long yield over the life of the short bond and the adjusted yield spread:

$$y_{t+1}^{(n-1)} - y_t^{(n)} = \beta_0 + \beta_1 \left(\frac{1}{n-1} (y_t^{(n)} - y_t^{(1)}) \right) + \varepsilon_{t+1}.$$

Under the Expectations Hypothesis, the null is $\beta_1 = 1$, which corresponds to α_1 in the instructions. Because the dependent variable uses overlapping horizons, HAC standard errors will be reported to account for serial correlation in the regression residuals.

b) One maturity OLS

First, we estimated a simple OLS regression with one maturity (m = 1 month), which gives the compact table reported below. This first step was useful to check the direction and magnitude of the relationship between the adjusted spread and future long-yield changes.

Table 9: Test 1: OLS regression of long yield changes on the adjusted spread

n	β_1	SE	$t(\beta_1 = 1)$	R^2	Obs.
3	-0.0753	0.5691	-1.8895	0.0001	774
6	-0.4989	0.5832	-2.5701	0.0043	774
9	-0.6652	0.6104	-2.7282	0.0058	774
12	-0.7620	0.6283	-2.8045	0.0061	774
24	-1.0534	0.6945	-2.9569	0.0069	774
36	-1.4109	0.7619	-3.1643	0.0089	774
48	-1.7538	0.7983	-3.4495	0.0108	774
60	-1.9096	0.8395	-3.4657	0.0105	774
120	-3.1495	1.0045	-4.1309	0.0153	652

The estimated slope coefficients are negative and far from 1. So even this simple specification already points to a rejection of the theory. However, this regression is only a reduced version of the full Campbell and Shiller setup, which requires several maturity pairs to reproduce Table 1(a) properly.

c) Replication of Campbell and Shiller's Table

This table reproduces the structure of Table 1(a) in Campbell and Shiller (1991) using the full monthly sample available in my data. Each cell reports the estimated slope coefficient and its standard error for a given pair of maturities (n, m).

Table 10: Regression of long yield changes on the adjusted spread

$n\backslash m$	1 m	2 m	3 m	4 m	6 m	12 m	24 m	60 m
	0.1679							
2	(0.5744)	–	–	–	–	–	–	–
	-0.0753	-0.6086						
3	(0.5691)	(0.9574)	–	–	–	–	–	–
	-0.2650	-0.8341	-0.7710					
4	(0.5692)	(0.9719)	(1.2548)	–	–	–	–	–
	-0.4989	-1.1722	-1.2972	-1.1754				
6	(0.5832)	(1.0138)	(1.3072)	(1.5227)	–	–	–	–
	-0.6652	-1.5457	-1.8912	-1.9184	-1.4975			
9	(0.6104)	(1.0825)	(1.4255)	(1.6887)	(2.3667)	–	–	–
	-0.7620	-1.8599	-2.3875	-2.6143	-2.5764			
12	(0.6283)	(1.1315)	(1.5286)	(1.8314)	(2.5233)	–	–	–
	-1.0534	-2.4132	-3.1889	-3.9216	-4.9836	-6.1660		
24	(0.6945)	(1.2504)	(1.7119)	(2.0604)	(2.8158)	(5.3291)	–	–
	-1.4109	-3.0018	-4.0347	-4.9680	-6.4750	-9.6136	2.0545	
36	(0.7619)	(1.3868)	(1.8916)	(2.2688)	(3.0352)	(5.9828)	(14.5506)	–
	-1.7538	-3.6150	-4.8459	-6.0999	-8.3448	-13.4362	-5.3956	
48	(0.7983)	(1.4618)	(1.9907)	(2.3733)	(3.2507)	(6.4828)	(14.5258)	–
	-1.9096	-3.8892	-5.1639	-6.6032	-9.0579	-15.1807	-10.5653	
60	(0.8395)	(1.5149)	(2.0568)	(2.4736)	(3.4385)	(6.8717)	(14.7561)	–
	-3.1495	-6.0928	-8.0180	-9.8921	-13.6796	-25.4702	-30.1485	-33.9275
120	(1.0045)	(1.8730)	(2.5308)	(3.0966)	(4.3880)	(8.4250)	(16.4213)	(32.8909)

The results show that the coefficients are generally negative and far from 1, which indicates that the yield spread does not have the predictive power implied by the Expectations Hypothesis. Moreover, the estimated coefficients become more negative as maturity increases, meaning that a high spread does not predict a rise in the long yield; instead, it tends to predict the opposite direction. Overall, the evidence strongly rejects the Expectations Hypothesis.

The standard errors also increase with maturity, especially for the longest horizons. This is consistent with the fact that the regression residuals are affected by overlapping observations, which reduces the amount of independent information in the sample. As a result, long-horizon estimates are less precise.

3.2 Test 2: Short yield changes over the life of the long bond

a) Methodology

Campbell and Shiller (1991) propose a second test of the Expectations Hypothesis based on short-yield changes over the life of the long bond. Starting from the identity in the assignment, we estimate the regression:

$$\frac{1}{n-1}\sum_{i=0}^{n-1} y_{1,t+i} - y_{1t} = \beta_0 + \beta_1(y_{nt} - y_{1t}) + \varepsilon_{t+n-1}.$$

Under the Expectations Hypothesis, the null hypothesis is $\beta_1 = 1$ and $\beta_0 = -\Phi_n$.
As in Test 1, the dependent variable uses overlapping horizons, so HAC standard errors are reported to account for serial correlation in the regression residuals

b) Results

Table 11: Test 2: regression coefficients

	1 m	2 m	3 m	4 m	6 m	12 m	24 m	60 m
	1.1679							
2	(0.5744)							
	1.2807	0.9277						
3	(0.7494)	(0.5834)						
	1.6187	1.0692	0.7320					
4	(0.9256)	(0.7708)	(0.5945)					
	2.4431	1.8307	1.2201	0.7578				
6	(1.2179)	(1.0932)	(0.9704)	(0.8145)				
	3.7997	3.1373	2.5456	1.9035	0.9319			
9	(1.7330)	(1.5755)	(1.4322)	(1.3019)	(1.0316)			
	5.6451	4.6922	3.8247	3.0854	2.0945			
12	(2.2825)	(2.0893)	(1.9234)	(1.7839)	(1.5572)			
	13.8191	12.4639	11.0792	9.7430	7.4176	3.3375		
24	(5.0281)	(4.8493)	(4.6517)	(4.4476)	(4.0333)	(2.9664)		
	23.8635	22.4656	20.9092	19.3183	16.4220	9.7373	1.3963	
36	(8.3815)	(8.3582)	(8.2956)	(8.1948)	(7.8969)	(6.4471)	(3.6946)	
	33.6742	32.8848	32.0697	31.1264	28.7512	20.4112	5.9877	
48	(10.5553)	(10.4835)	(10.4209)	(10.3852)	(10.3842)	(10.0482)	(7.6863)	
	43.9437	43.1356	42.2901	41.3763	39.5353	33.8727	17.5106	
60	(12.3007)	(12.2060)	(12.0931)	(11.9820)	(11.8026)	(11.6803)	(11.2320)	
	93.3786	92.0641	90.5051	88.8551	85.5376	77.1479	60.3227	23.7002
120	(22.3296)	(22.4214)	(22.4811)	(22.5070)	(22.4608)	(22.0153)	(22.0579)	(18.3383)

The estimates in Table 11 are generally positive and increase with maturity, while their dispersion also rises sharply at longer horizons, making inference less precise. This suggests that the expectations hypothesis performs poorly, which is broadly consistent with Campbell and Shiller's findings.

References

Campbell, John Y. and Robert J. Shiller (1991). "Yield Spreads and Interest Rate Movements: A Bird's Eye View". In: *The Review of Economic Studies* 58.3, pp. 495–514.

Markowitz, Harry (1952). "Portfolio Selection". In: *The Journal of Finance* 7.1, pp. 77–91.

4 AI assistance

This report was supported by the use of artificial intelligence tools to improve clarity, formatting, and overall presentation. These tools were used in a limited and supportive way, mainly for language correction, refinement of academic writing, and LaTeX formatting. All analyses, interpretations, and conclusions were developed independently by the author.

- Review the text to correct grammar, spelling, and improve overall clarity and readability.
- Improve the academic tone and make sentences more concise and precise without altering the meaning.
- Suggest improvements for LaTeX formatting and document structure.
- Check the consistency of terminology and concepts throughout the report.
- Identify unclear or redundant sentences and suggest improvements.
- Verify that arguments and conclusions remain coherent across sections.
- Ensure that the analysis is logically structured and that each section contributes clearly to the overall reasoning.

5 Appendices

Here are all the codes used in the making of this project (Python). The complete implementation with outputs is available in the attached .ipynb file.

Q1 — Time Series

Q1.1.c.i — Conditional mean $E_1[x_t]$

```
E_x = []

for i in t:
    E_x.append( x*(rho**(i-1)) )
```

```
plt.plot(t, E_x, marker='o')
plt.ylabel('E[x]')
plt.xlabel('Time')
plt.title('Evolution of the conditional mean')
```

Q1.1.c.ii — Conditional mean ± 1 std

```
t
```

```
range(1, 21)
```

```
VAR_x = []

for j in t:
    VAR_x.append(sigma**2 * (1 - rho**(2*(j-1))) / (1 - rho**2))
```

```
#conversion needed
E_x = np.array(E_x)
VAR_x = np.array(VAR_x)

E_x_plus_VAR = E_x + np.sqrt(VAR_x)
E_x_minus_VAR = E_x - np.sqrt(VAR_x)
```

```
plt.plot(t, E_x)
plt.plot(t, E_x_plus_VAR, color='r', linestyle='dashed')
plt.plot(t, E_x_minus_VAR, color='r', linestyle='dashed')
plt.scatter(1, 1, zorder=5)
plt.axhline(y=0, color='black')
plt.ylabel('E[x]')
plt.xlabel('Time')
plt.title('Evolution of the conditional mean ±1 std')
```

Q1.1.c.iii — Prediction cones: Chebychev vs Normality

Normality assumption

```
normal_band_90_plus = E_x + 1.645 * np.sqrt(VAR_x)
normal_band_90_minus = E_x - 1.645 * np.sqrt(VAR_x)
plt.plot(t, E_x)
plt.plot(t, normal_band_90_plus, color='r', linestyle='dashed')
plt.plot(t, normal_band_90_minus, color='r', linestyle='dashed')
plt.axhline(y=0, color='black')
plt.ylabel('E[x]')
plt.xlabel('Time')
plt.title('Prediction cone under normality (90%)')
```

Afficher la sortie masquée

```
normal_band_95_plus = E_x + 1.96 * np.sqrt(VAR_x)
normal_band_95_minus = E_x - 1.96 * np.sqrt(VAR_x)
plt.plot(t, E_x)
plt.plot(t, normal_band_95_plus, color='darkred', linestyle='dashed')
plt.plot(t, normal_band_95_minus, color='darkred', linestyle='dashed')
plt.axhline(y=0, color='black')
plt.ylabel('E[x]')
plt.xlabel('Time')
plt.title('Prediction cone under normality (95%)')
```

Afficher la sortie masquée

```
plt.plot(t, E_x)
plt.plot(t, normal_band_90_plus, color='r', linestyle='dashed')
plt.plot(t, normal_band_95_plus, color='darkred', linestyle='dashed')
plt.plot(t, normal_band_90_minus, color='r', linestyle='dashed')
plt.plot(t, normal_band_95_minus, color='darkred', linestyle='dashed')
plt.axhline(y=0, color='black')
plt.ylabel('E[x]')
plt.xlabel('Time')
plt.legend(['Conditional mean','90%', '95%'])
plt.title('Prediction cones comparison under normality')
```

Afficher la sortie masquée

Chebychev Inequality

```
k_90=np.sqrt(10)
chebychev_band_90_plus = E_x + k_90 * np.sqrt(VAR_x)
chebychev_band_90_minus = E_x - k_90 * np.sqrt(VAR_x)
plt.plot(t, E_x)
plt.plot(t, chebychev_band_90_plus, color='r', linestyle='dashed')
plt.plot(t, chebychev_band_90_minus, color='r', linestyle='dashed')
plt.axhline(y=0, color='black')
plt.ylabel('E[x]')
plt.xlabel('Time')
plt.title('Prediction cone under Chebychev inequality (90%)')
```

Afficher la sortie masquée

```
k_95=np.sqrt(20)
chebychev_band_95_plus = E_x + k_95 * np.sqrt(VAR_x)
chebychev_band_95_minus = E_x - k_95 * np.sqrt(VAR_x)
plt.plot(t, E_x)
plt.plot(t, chebychev_band_95_plus, color='darkred', linestyle='dashed')
plt.plot(t, chebychev_band_95_minus, color='darkred', linestyle='dashed')
plt.axhline(y=0, color='black')
plt.ylabel('E[x]')
plt.xlabel('Time')
plt.title('Prediction cone under Chebychev inequality (95%)')
```

Afficher la sortie masquée

```
plt.plot(t, E_x)
plt.plot(t, chebychev_band_90_plus, color='r', linestyle='dashed')
plt.plot(t, chebychev_band_95_plus, color='darkred', linestyle='dashed')
plt.plot(t, chebychev_band_90_minus, color='r', linestyle='dashed')
plt.plot(t, chebychev_band_95_minus, color='darkred', linestyle='dashed')
plt.axhline(y=0, color='black')
plt.ylabel('E[x]')
plt.xlabel('Time')
plt.legend(['Conditional mean','90%', '95%'])
plt.title('Prediction cones comparison under Chebychev')
```

Afficher la sortie masquée

Q1.1.c.iv — AR(1) path simulations

iv. Path simulation

```
T=20
sigma = 0.25
rho = 0.95
```

```
eps = []
for s in range(20):
    eps.append(np.random.normal(0, sigma, size=T))
```

```
for d in range(20):
    eps[d][0] = 0
```

```
paths = []

for e in range(20):
    x = np.empty(T)
    x[0] = 1
    for t in range(0, T-1):        # t = 0, 1, ..., 18
        x[t+1] = rho * x[t] + eps[e][t+1]
    paths.append(x.copy())
```

```
paths = np.array(paths).T
```

Q1.2 — Long-horizon returns

Q1.2.h.i.B — Impulse-response function MA(1)

```
T=5
```

```
def MA(T, delta, color):
    r = [0]*T
    e = [0]*T
    e[1] = 1

    for t in range(1,T):
        r[t] = e[t] + delta*e[t-1]

    p = np.cumsum(r)

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12,4))

    ax1.plot(r, color=color, marker='o')
    ax1.set_xlabel('t')
    ax1.set_ylabel('r_t')
    ax1.set_title(f'Impulse-response function with  δ={delta}')

    ax2.plot(p, color=color, marker='o')
    ax2.set_xlabel('t')
    ax2.set_ylabel('p_t')
    ax2.set_title(f'Cumulative log return function with  δ={delta}')

    print(r)
    print(p)

    return r, p
```

```
r_momentum, p_momentum = MA(T, delta = 0.5, color = 'C0')
```

Afficher la sortie masquée

```
r_iid, p_iid = MA(T, delta = 00, color = 'C1')
```

Afficher la sortie masquée

```
r_mean_rev, p_mean_rev = MA(T, delta = -0.5, color = 'C2')
```

Afficher la sortie masquée

Q3.1 — Stock Market: Efficient Frontier

Q3.1.1 — Data

```
tickers = ["VEUSX", "VBLIX"]
#VEUSX - Vanguard European Stock Index Fund
#VBLIX - Vanguard Long-Term Bond Fund
raw = yf.download(tickers, start="2015-01-01", auto_adjust=True)["Close"]

#Prices
prices_m = raw.resample("ME").last().dropna()

#Normalized
prices_norm = prices_m / prices_m.iloc[0] * 100

#Returns
ret_m = prices_m.pct_change().dropna()
```

```
[**********100%**********]  2 of 2 completed
```

Q3.1.2 — Normalized price chart

```
plt.figure(figsize=(12, 5))
plt.plot(prices_norm)
plt.axvspan("2020-02-01", "2020-06-30", color="blue", alpha=0.2, label="COVID-19")
plt.axvspan("2022-02-01", "2022-12-31", color="red", alpha=0.15, label="War in Ukraine")
plt.legend(prices_norm)
plt.title("Normalized price chart")
plt.xlabel("time")
plt.ylabel("%")
```

Afficher la sortie masquée

Q3.1.3 — Sample moments

```
mu  = ret_m.mean()
cov = ret_m.cov()
std = ret_m.std()
```

```
corr = ret_m.corr()
cov_100  = ret_m.cov() * 100  # en % pour lisibilité

fig, axes = plt.subplots(1, 2, figsize=(10, 3.5))

sns.heatmap(corr, annot=True, fmt=".4f", cmap="coolwarm", center=0,
            vmin=-1, vmax=1, ax=axes[0], linewidths=0.5)
axes[0].set_title("Correlation Matrix")

sns.heatmap(cov_100, annot=True, fmt=".6f", cmap="Blues",
            ax=axes[1], linewidths=0.5)
axes[1].set_title("Variance-Covariance Matrix (monthly)")

plt.tight_layout()
plt.savefig("heatmaps.png", dpi=150)
plt.show()
```

Afficher la sortie masquée

Q3.1.4 — Efficient frontier (2 risky assets)

```python
mu  = ret_m.mean().values        # shape (2,)
cov = ret_m.cov().values         # shape (2,2)

# Efficient frontier
weights = np.linspace(-1, 1.5, 500)

port_ret = []
port_std = []

for w in weights:
    wvec     = np.array([w, 1 - w])
    port_ret.append(wvec @ mu)
    port_std.append(np.sqrt(wvec @ cov @ wvec))

port_ret = np.array(port_ret)
port_std = np.array(port_std)

# Plot
fig, ax = plt.subplots(figsize=(9, 6))

ax.plot(port_std * 100, port_ret * 100, color="steelblue", lw=2, label="Efficient Frontier")

# Individual assets
for name, m, s in zip(["VBLIX", "VEUSX"], mu, np.sqrt(np.diag(cov))):
    ax.scatter(s * 100, m * 100, zorder=5, s=60)
    ax.annotate(name, (s * 100, m * 100), textcoords="offset points", xytext=(8, 4))

ax.set_xlabel("Monthly Std Dev (%)")
ax.set_ylabel("Monthly Mean Return (%)")
ax.set_title("Efficient Frontier — Two Risky Assets")
ax.legend()
plt.tight_layout()
plt.show()
```

Afficher la sortie masquée

Q3.1.5 — Global minimum variance portfolio

```python
# Min var pf
w_min_var = (cov[1,1] - cov[0,1]) / (cov[0,0] + cov[1,1] - 2*cov[0,1])
ret_min_var = np.array([w_min_var, 1-w_min_var]) @ mu
std_min_var = np.sqrt(np.array([w_min_var, 1-w_min_var]) @ cov @ np.array([w_min_var, 1-w_min_var]))
sharpe_min_var = (ret_min_var - 0.00025) / std_min_var

print(f"MV weights  : VBLIX={w_min_var:.4f}, VEUSX={1-w_min_var:.4f}")
print(f"MV mean     : {ret_min_var*100:.4f}%")
print(f"MV std      : {std_min_var*100:.4f}%")
print(f"MV Sharpe   : {sharpe_min_var:.4f}")

#yearly data
ret_min_var_yr = ret_min_var*12
std_min_var_yr = std_min_var*np.sqrt(12)
sharpe_min_var_yr = (ret_min_var_yr - 0.00025*12) / std_min_var_yr

print(f"Yearly MV mean     : {ret_min_var_yr*100:.4f}%")
print(f"Yearly MV std      : {std_min_var_yr*100:.4f}%")
print(f"Yearly MV Sharpe   : {sharpe_min_var_yr:.4f}")
```

```
MV weights  : VBLIX=0.8148, VEUSX=0.1852
MV mean     : 0.2368%
MV std      : 3.1090%
MV Sharpe   : 0.0681
Yearly MV mean     : 2.8411%
Yearly MV std      : 10.7698%
Yearly MV Sharpe   : 0.2359
```

```python
fig, ax = plt.subplots(figsize=(9, 6))

ax.plot(port_std * 100, port_ret * 100, color="steelblue", lw=2, label="Efficient Frontier")
#Min_var
ax.scatter(std_min_var * 100, ret_min_var * 100, color="red", zorder=5, s=80, label="Min Variance Portfolio")

# Individual assets
for name, m, s in zip(["VBLIX", "VEUSX"], mu, np.sqrt(np.diag(cov))):
    ax.scatter(s * 100, m * 100, zorder=5, s=60)
    ax.annotate(name, (s * 100, m * 100), textcoords="offset points", xytext=(8, 4))

ax.set_xlabel("Monthly Std Dev (%)")
ax.set_ylabel("Monthly Mean Return (%)")
ax.set_title("Efficient Frontier — Two Risky Assets")
ax.legend()
plt.tight_layout()
plt.show()
```

Afficher la sortie masquée

Q3.1.6 — Tangency portfolio

```
from scipy.optimize import minimize
```

```
rf = 0.00025

excess_mu = mu - rf

inv_cov   = np.linalg.inv(cov)
ones      = np.ones(len(mu))

w_tan_raw    = inv_cov @ excess_mu
w_tangency   = w_tan_raw / (ones @ w_tan_raw)  # weights sum to 1

ret_tangency  = w_tangency @ mu
std_tangency  = np.sqrt(w_tangency @ cov @ w_tangency)  # ← était w_tan
sharp_tangency = (ret_tangency - rf) / std_tangency      # ← était tan_ret/tan_std

print(f"Tangency weights : VBLIX={w_tangency[0]:.4f}, VEUSX={w_tangency[1]:.4f}")  # ← était w_tan[1]
print(f"Tangency mean    : {ret_tangency*100:.4f}%")
print(f"Tangency std     : {std_tangency*100:.4f}%")
print(f"Sharpe ratio     : {sharp_tangency:.4f}")

# Annualized
ret_tangency_yr   = ret_tangency * 12
std_tangency_yr   = std_tangency * np.sqrt(12)
sharp_tangency_yr = (ret_tangency_yr - rf * 12) / std_tangency_yr

print(f"Yearly Tangency mean   : {ret_tangency_yr*100:.4f}%")
print(f"Yearly Tangency std    : {std_tangency_yr*100:.4f}%")
print(f"Yearly Tangency Sharpe : {sharp_tangency_yr:.4f}")
```

```
Tangency weights : VBLIX=-0.5425, VEUSX=1.5425
Tangency mean    : 1.0671%
Tangency std     : 6.8969%
Sharpe ratio     : 0.1511
Yearly Tangency mean   : 12.8054%
Yearly Tangency std    : 23.8916%
Yearly Tangency Sharpe : 0.5234
```

```
fig, ax = plt.subplots(figsize=(9, 6))

ax.plot(port_std * 100, port_ret * 100, color="steelblue", lw=2, label="Efficient Frontier")
#Min_var
ax.scatter(std_min_var * 100, ret_min_var * 100, color="red", zorder=5, s=80, label="Min Variance Portfolio")
#Tangency
ax.scatter(std_tangency * 100, ret_tangency * 100, color="green", zorder=5, s=80, label="Tangency Portfolio")

# Individual assets
for name, m, s in zip(["VBLIX", "VEUSX"], mu, np.sqrt(np.diag(cov))):
    ax.scatter(s * 100, m * 100, zorder=5, s=60)
    ax.annotate(name, (s * 100, m * 100), textcoords="offset points", xytext=(8, 4))

ax.set_xlabel("Monthly Std Dev (%)")
ax.set_ylabel("Monthly Mean Return (%)")
ax.set_title("Efficient Frontier — Two Risky Assets")
ax.legend()
plt.tight_layout()
plt.show()
```

Afficher la sortie masquée

Q3.2 — Stock Market: Industry Momentum

Q3.2.0 — Data import and treatment

```
def clean_datam(df, columns):
  df[0] = pd.to_datetime(df[0], format='%Y%m', errors='coerce')
  df = df.dropna()
  df.columns = columns
  df = df.set_index('Date')
  return df
```

```
df_30 = pd.read_csv(r'data\30_Industry_Portfolios.csv',
                    header=None,
                    skiprows=12,
                    nrows=1208-12)
```

```
df_30 =clean_datam(df_30, ['Date','Food', 'Beer','Smoke','Games','Books','Hshld','Clths','Hlth','Chems','Txtls','C
```

```
df_factors = pd.read_csv('data/F-F_Research_Data_Factors.csv',
                         header=None,
                         skiprows=5,  #Skip header
                         nrows=1201-5)
```

```
df_factors = clean_datam(df_factors, ['Date','Mkt-RF','SMB','HML','RF'])
```

Q3.2.1 — Industry ranks

```
rolling_returns = df_30.rolling(window=12).mean().shift(1)
rolling_returns = rolling_returns['1927-07':]
```

```
ranks = rolling_returns.rank(axis=1)
mean_ranks = ranks.mean()
```

```
print("Lowest average rank:", mean_ranks.idxmin(), mean_ranks.min())
print("Highest average rank:", mean_ranks.idxmax(), mean_ranks.max())
```

```
Lowest average rank: Telcm 14.152027027027026
Highest average rank: BusEq 16.88597972972973
```

```
mean_ranks = ranks.mean().sort_values()

plt.figure(figsize=(14,6))
plt.bar(mean_ranks.index, mean_ranks.values)
plt.xticks(rotation=90)
plt.ylabel('Mean rank')
plt.title('Mean rank by industry')
plt.tight_layout()
plt.show()
```

Afficher la sortie masquée

```
mean_ranks.head
```

```
plt.figure(figsize=(12,5))
plt.plot(ranks['Oil'], label='Mean Evolution')
plt.axhline(ranks['Oil'].mean(), color = 'r', label='Ranks mean')
plt.title("Stability of Oil Industry")
plt.xlabel('Time')
plt.ylabel('Rank')
plt.legend(loc="upper left")
```

Afficher la sortie masquée

```
rank_autocorr = ranks.corrwith(ranks.shift(1), axis=1)

print("Average rank autocorrelation:", rank_autocorr.mean())
```

```
Average rank autocorrelation: 0.9103344317490775
```

```
#Top 5 Turnover
top5 = ranks >= 26

# change from one month to another
top5_changes = (top5 != top5.shift(1)).sum(axis=1)

#mean turnover
avg_top5_turnover = (top5_changes / 5).mean()

print("Average top-5 turnover:", avg_top5_turnover)
```

```
Average top-5 turnover: 0.4489864864864865
```

Q3.2.2 — Winner portfolio

```
#Id winners
winners = (ranks > 15).astype(int)
#creates weights
winner_weights = winners.div(winners.sum(axis=1), axis=0)
#Returns
winner_ret = (winner_weights * df_30.loc[winner_weights.index]).sum(axis=1)
```

```
print("Index matches:", winner_weights.index.equals(df_30.loc[winner_weights.index].index))
```

```
Index matches: True
```

```
#Risk free
df_factors['RF'].mean()
```

```
np.float64(0.270066889632107)
```

```
winner_excess_rets = winner_ret - df_factors['RF']
print("Winner excess mean:", winner_excess_rets.mean())
print("Winner excess std:",winner_excess_rets.std())
```

```
Winner excess mean: 0.964236451295045
Winner excess std: 5.5618922256745265
```

```
winner_sharpe_monthly = winner_excess_rets.mean() / winner_excess_rets.std()
winner_sharpe_annualized = winner_sharpe_monthly * np.sqrt(12)

print(f"Monthly Sharpe Ratio: {winner_sharpe_monthly:.4f}")
print(f"Annualized Sharpe Ratio: {winner_sharpe_annualized:.4f}")
```

```
Monthly Sharpe Ratio: 0.1734
Annualized Sharpe Ratio: 0.6006
```

Q3.2.3 — Loser portfolio

```
#Id loosers
loosers = (ranks < 15).astype(int)
#creates weights
looser_weights = loosers.div(loosers.sum(axis=1), axis=0)
#Returns
looser_ret = (looser_weights * df_30.loc[looser_weights.index]).sum(axis=1)
```

```
print("Index matches:", looser_weights.index.equals(df_30.loc[looser_weights.index].index))
```

```
Index matches: True
```

```
looser_excess_rets = looser_ret - df_factors['RF']

print("Looser excess mean:", looser_excess_rets.mean())
print("Looser excess std:",looser_excess_rets.std())
```

```
Looser excess mean: 0.5170101351351352
Looser excess std: 6.0013552732679045
```

```
looser_sharpe_monthly = looser_excess_rets.mean() / looser_excess_rets.std()
looser_sharpe_annualized = looser_sharpe_monthly * np.sqrt(12)

print(f"Monthly Sharpe Ratio: {looser_sharpe_monthly:.4f}")
print(f"Annualized Sharpe Ratio: {looser_sharpe_annualized:.4f}")
```

```
Monthly Sharpe Ratio: 0.0861
Annualized Sharpe Ratio: 0.2984
```

```
market_ret = df_factors['Mkt-RF']['1927-07':]

market_excess_rets = market_ret + df_factors['RF']['1927-07':]

market_sharpe_monthly = market_excess_rets.mean() / market_excess_rets.std()
market_sharpe_annualized = market_sharpe_monthly * np.sqrt(12)

print(f"Monthly Sharpe Ratio: {market_sharpe_monthly:.4f}")
print(f"Annualized Sharpe Ratio: {market_sharpe_annualized:.4f}")
```

```
Monthly Sharpe Ratio: 0.1796
Annualized Sharpe Ratio: 0.6220
```

+ Code + Texte

Q3.2.4 — Cumulative returns

```
longshort_rets = winner_ret - looser_ret
```

```
def compute_cumulative(returns):
    # Produit cumulé : (1 + r1) * (1 + r2) * ...
    return (1 + returns/100).cumprod()
```

```
winner_cumulative_ret = compute_cumulative(winner_ret)
looser_cumulative_ret = compute_cumulative(looser_ret)
market_cumulative_ret = compute_cumulative(market_ret)
longshort_cumulative_ret = compute_cumulative(longshort_rets)
```

```
[ ]
    plt.figure(figsize=(15, 8))
    plt.plot(winner_cumulative_ret, label='Winner')
    plt.plot(looser_cumulative_ret, label='Looser')
    plt.plot(market_cumulative_ret, label='Market')
    plt.plot(longshort_cumulative_ret, label='Long-Short')

    plt.yscale('log')
    plt.title('Cumulative Returns (Log Scale)')
    plt.legend()
    plt.grid(alpha=0.3)
    plt.show()
```

> Afficher la sortie masquée

Q3.2.5 — Ind-mom loss episodes

```
[ ]
def plot_zoom_crash(cumulative_series, start_date, end_date, title="Zoom on Period"):
    zoom = cumulative_series.loc[start_date:end_date]
    if zoom.empty:
        print("Période vide, vérifie tes dates.")
        return

    norm = zoom / zoom.iloc[0]

    # Plot
    plt.figure(figsize=(10, 5))
    plt.plot(norm.index, norm, linewidth=2, color='C3')

    plt.title(f"{title} ({start_date} to {end_date})")
    plt.ylabel("Performance relative (Base 1)")
    plt.axhline(1, color='black', linewidth=0.5, linestyle='--')
    plt.grid(alpha=0.3)


    plt.tight_layout()
    plt.show()
```

```
[ ]
plot_zoom_crash(longshort_cumulative_ret, '2008-01-01', '2010-06-01', title="Zoom on Period")
```

Afficher la sortie masquée

```
[ ]
print(longshort_rets['2009-03-01':'2009-05-01'])
print(winner_ret['2009-03-01':'2009-05-01'])
print(looser_ret['2009-03-01':'2009-05-01'])
```

```
Date
2009-03-01    -4.208000
2009-04-01   -19.164238
2009-05-01    -4.859667
dtype: float64
Date
2009-03-01    7.482000
2009-04-01    7.589333
2009-05-01    3.315333
dtype: float64
Date
2009-03-01    11.690000
2009-04-01    26.753571
2009-05-01     8.175000
dtype: float64
```

```
[ ]
plot_zoom_crash(longshort_cumulative_ret, '2015-08-01', '2016-12-01', title="Zoom on Period")
```

Afficher la sortie masquée

```
[ ]
print(longshort_rets['2016-02-01':'2016-04-01'])
print(winner_ret['2016-02-01':'2016-04-01'])
print(looser_ret['2016-02-01':'2016-04-01'])
```

```
Date
2016-02-01   -3.908429
2016-03-01   -4.496952
2016-04-01   -6.890476
dtype: float64
Date
2016-02-01    0.328000
2016-03-01    6.047333
2016-04-01   -0.613333
dtype: float64
Date
2016-02-01     4.236429
2016-03-01    10.544286
2016-04-01     6.277143
dtype: float64
```

Q4 — Bond Market: Expectation Hypothesis

Q4.0 — Data import

```
[ ]
    df = pd.read_csv('data\LW_monthly.csv', skiprows=8)  # Skip headers
    df.columns = [col.strip() for col in df.columns]  # Clean
    df['date'] = pd.to_datetime(df.iloc[:,0].astype(str), format='%Y%m')
    df.set_index('date', inplace=True)
    df = df.iloc[:,1:].astype(float)  # Yields float
```

Q4.1 — Test 1: Long yield changes

Test for 1 variable

```
[ ]
    # n = 12 (1 yr)
    n = 12
    col_n = '12 m'
    col_n_minus_1 = '11 m'
    col_1 = '1 m'

    # dependent Variable : y_{t+1}^{(n-1)} - y_t^{(n)}
    # décalle n-1 de 1 mois
    delta_y = df[col_n_minus_1].shift(-1) - df[col_n]

    # explicative Variable = (1/(n-1)) * (y_t^{(n)} - y_t^{(1)})
    spread_adj = (df[col_n] - df[col_1]) / (n - 1)

    # cleaning
    data = pd.concat([delta_y, spread_adj], axis=1).dropna()
    data.columns = ['delta_y', 'spread_adj']
```

```
[ ]
    X = sm.add_constant(data['spread_adj'])
    model = sm.OLS(data['delta_y'], X).fit(
        cov_type='HAC',
        cov_kwds={'maxlags': 12}
    )

    print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                delta_y   R-squared:                       0.006
Model:                            OLS   Adj. R-squared:                  0.005
Method:                 Least Squares   F-statistic:                     1.471
Date:                Tue, 05 May 2026   Prob (F-statistic):              0.226
Time:                        17:20:31   Log-Likelihood:                -444.91
No. Observations:                 774   AIC:                             893.8
Df Residuals:                     772   BIC:                             903.1
Df Model:                           1
Covariance Type:                  HAC
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.0102      0.025      0.405      0.686      -0.039       0.060
spread_adj    -0.7620      0.628     -1.213      0.225      -1.993       0.469
==============================================================================
Omnibus:                      332.745   Durbin-Watson:                   1.637
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            13332.915
Skew:                          -1.223   Prob(JB):                         0.00
Kurtosis:                      23.185   Cond. No.                         22.7
==============================================================================

Notes:
[1] Standard Errors are heteroscedasticity and autocorrelation robust (HAC) using 12 lags and without small sample
```

```python
mats = [3,6,9,12,24,36,48,60,120]
results = []

for n in mats:
    col_n = f'{n} m'
    col_n1 = f'{n-1} m'
    col_1 = '1 m'

    if col_n not in df.columns or col_n1 not in df.columns or col_1 not in df.columns:
        continue

    tmp = df[[col_n, col_n1, col_1]].dropna()
    tmp['delta_y'] = tmp[col_n1].shift(-1) - tmp[col_n]
    tmp['spread_adj'] = (tmp[col_n] - tmp[col_1]) / (n - 1)
    data = tmp[['delta_y', 'spread_adj']].dropna()

    X = sm.add_constant(data['spread_adj'])
    model = sm.OLS(data['delta_y'], X).fit(cov_type='HAC', cov_kwds={'maxlags':12})

    # Noms des paramètres au lieu des indices
    beta1 = model.params['spread_adj']
    se_beta1 = model.bse['spread_adj']
    t_beta1_1 = (beta1 - 1) / se_beta1

    results.append({
        'n': n,
        'beta1': beta1,
        'se': se_beta1,
        't(beta1=1)': t_beta1_1,
        'R2': model.rsquared,
        'obs': len(data)
    })

pd.DataFrame(results)
```

	n	beta1	se	t(beta1=1)	R2	obs
0	3	-0.075251	0.569052	-1.889549	0.000123	774
1	6	-0.498859	0.583195	-2.570080	0.004264	774
2	9	-0.665236	0.610375	-2.728216	0.005752	774
3	12	-0.761966	0.628261	-2.804512	0.006070	774
4	24	-1.053437	0.694459	-2.956886	0.006873	774
5	36	-1.410894	0.761909	-3.164280	0.008942	774
6	48	-1.753817	0.798328	-3.449483	0.010752	774
7	60	-1.909555	0.839536	-3.465670	0.010524	774
8	120	-3.149518	1.004511	-4.130884	0.015285	652

Test multivariate

```
results = []
mats_n = [2, 3, 4, 6, 9, 12, 24, 36, 48, 60, 120]
mats_m = [1, 2, 3, 4, 6, 12, 24, 60]

for n in mats_n:
    row = {'n': n}
    for m in mats_m:
        if m >= n: continue # m < n

        col_n = f'{n} m'
        col_n_minus_m = f'{n-m} m'
        col_m = f'{m} m'

        # Test : y(n-m, t+m) - y(n, t) = ...
        delta_y = df[col_n_minus_m].shift(-m) - df[col_n]
        spread_adj = (df[col_n] - df[col_m]) / (n - m)

        data = pd.concat([delta_y, spread_adj], axis=1).dropna()

        X = sm.add_constant(data[1]) # spread_adj
        model = sm.OLS(data[0], X).fit(cov_type='HAC', cov_kwds={'maxlags': 12})

        # format of beta (se)
        row[f'{m} m'] = f"{model.params[1]:.4f}\n({model.bse[1]:.4f})"

    results.append(row)

final_df = pd.DataFrame(results).set_index('n')
print(final_df)
```

```
                  1 m              2 m              3 m  \
n
2     0.1679\n(0.5744)              NaN              NaN
3    -0.0753\n(0.5691)  -0.6086\n(0.9574)              NaN
4    -0.2650\n(0.5692)  -0.8341\n(0.9719)  -0.7710\n(1.2548)
6    -0.4989\n(0.5832)  -1.1722\n(1.0138)  -1.2972\n(1.3072)
9    -0.6652\n(0.6104)  -1.5457\n(1.0825)  -1.8912\n(1.4255)
12   -0.7620\n(0.6283)  -1.8599\n(1.1315)  -2.3875\n(1.5286)
24   -1.0534\n(0.6945)  -2.4132\n(1.2504)  -3.1889\n(1.7119)
36   -1.4109\n(0.7619)  -3.0018\n(1.3868)  -4.0347\n(1.8916)
48   -1.7538\n(0.7983)  -3.6150\n(1.4618)  -4.8459\n(1.9907)
60   -1.9096\n(0.8395)  -3.8892\n(1.5149)  -5.1639\n(2.0568)
120  -3.1495\n(1.0045)  -6.0928\n(1.8730)  -8.0180\n(2.5308)

                  4 m              6 m              12 m  \
n
2                 NaN              NaN               NaN
3                 NaN              NaN               NaN
4                 NaN              NaN               NaN
6    -1.1754\n(1.5227)              NaN               NaN
9    -1.9184\n(1.6887)  -1.4975\n(2.3667)               NaN
12   -2.6143\n(1.8314)  -2.5764\n(2.5233)               NaN
24   -3.9216\n(2.0604)  -4.9836\n(2.8158)  -6.1660\n(5.3291)
36   -4.9680\n(2.2688)  -6.4750\n(3.0352)  -9.6136\n(5.9828)
48   -6.0999\n(2.3733)  -8.3448\n(3.2507)  -13.4362\n(6.4828)
60   -6.6032\n(2.4736)  -9.0579\n(3.4385)  -15.1807\n(6.8717)
120  -9.8921\n(3.0966)  -13.6796\n(4.3880)  -25.4702\n(8.4250)

                  24 m              60 m
n
2                  NaN               NaN
3                  NaN               NaN
4                  NaN               NaN
6                  NaN               NaN
9                  NaN               NaN
12                 NaN               NaN
24                 NaN               NaN
36    2.0545\n(14.5506)               NaN
48   -5.3956\n(14.5258)               NaN
60   -10.5653\n(14.7561)              NaN
120  -30.1485\n(16.4213)  -33.9275\n(32.8909)
```

Q4.2 — Test 2: Short yield changes

```
[ ]
results_comp = []
mats_n = [2, 3, 4, 6, 9, 12, 24, 36, 48, 60, 120]

for n in mats_n:
    m = 1
    col_n = f'{n} m'
    col_m = '1 m'
    if col_n not in df.columns or col_m not in df.columns:
        continue

    dep = sum(df[col_m].shift(-i) for i in range(1, n - m + 1)) / (n - m) - df[col_m]
    spread = (df[col_n] - df[col_m]) / (n - m)

    data = pd.concat([dep.rename('y'), spread.rename('x')], axis=1).dropna()
    if len(data) < 20:
        continue

    X = sm.add_constant(data['x'])
    model = sm.OLS(data['y'], X).fit(cov_type='HAC', cov_kwds={'maxlags': 12})

    results_comp.append({
        'n': n,
        'beta0': model.params['const'],
        'beta1': model.params['x'],
        'se_beta1': model.bse['x'],
        't(beta1=1)': (model.params['x'] - 1) / model.bse['x'],
        'R2': model.rsquared,
        'obs': int(model.nobs)
    })

comp_df = pd.DataFrame(results_comp).set_index('n')
print(comp_df)
```

```
           beta0      beta1   se_beta1  t(beta1=1)        R2  obs
n
2   -0.075368   1.167869   0.574408    0.292246  0.029459  774
3   -0.079238   1.280711   0.749427    0.374567  0.024356  773
4   -0.095933   1.618677   0.925572    0.668427  0.028013  772
6   -0.130347   2.443133   1.217922    1.184914  0.036335  770
9   -0.171443   3.799688   1.732951    1.615561  0.046700  767
12  -0.221849   5.645125   2.282495    2.035108  0.062012  764
24  -0.367849  13.819118   5.028070    2.549511  0.084753  752
36  -0.535693  23.863536   8.381454    2.727872  0.106121  740
48  -0.703073  33.674209  10.555275    3.095534  0.127125  728
60  -0.873878  43.943657  12.300689    3.491159  0.158649  716
120 -1.908546  93.378600  22.329630    4.137041  0.227776  534
```

```
[ ]
results2 = []
mats_n = [2, 3, 4, 6, 9, 12, 24, 36, 48, 60, 120]
mats_m = [1, 2, 3, 4, 6, 12, 24, 60]

for n in mats_n:
    row = {'n': n}
    for m in mats_m:
        if m >= n:
            continue

        col_n = f'{n} m'
        col_m = f'{m} m'

        if col_n not in df.columns or col_m not in df.columns:
            row[f'{m} m'] = ''
            continue

        dep = sum(df[col_m].shift(-i) for i in range(1, n - m + 1)) / (n - m) - df[col_m]
        x = (df[col_n] - df[col_m]) / (n - m)

        data = pd.concat([dep.rename('y'), x.rename('x')], axis=1).dropna()
        if len(data) < 20:
            row[f'{m} m'] = ''
            continue

        X = sm.add_constant(data['x'])
        model2 = sm.OLS(data['y'], X).fit(cov_type='HAC', cov_kwds={'maxlags': 12})
        row[f'{m} m'] = f"{model2.params['x']:.4f}\n({model2.bse['x']:.4f})"

    results2.append(row)

final_df2 = pd.DataFrame(results2).set_index('n')
print(final_df2)
```

```
                   1 m               2 m               3 m  \
n
2      1.1679\n(0.5744)               NaN               NaN
3      1.2807\n(0.7494)    0.9277\n(0.5834)               NaN
4      1.6187\n(0.9256)    1.0692\n(0.7708)    0.7320\n(0.5945)
6      2.4431\n(1.2179)    1.8307\n(1.0932)    1.2201\n(0.9704)
9      3.7997\n(1.7330)    3.1373\n(1.5755)    2.5456\n(1.4322)
12     5.6451\n(2.2825)    4.6922\n(2.0893)    3.8247\n(1.9234)
24    13.8191\n(5.0281)   12.4639\n(4.8493)   11.0792\n(4.6517)
36    23.8635\n(8.3815)   22.4656\n(8.3582)   20.9092\n(8.2956)
48   33.6742\n(10.5553)  32.8848\n(10.4835)  32.0697\n(10.4209)
60   43.9437\n(12.3007)  43.1356\n(12.2060)  42.2901\n(12.0931)
120  93.3786\n(22.3296)  92.0641\n(22.4214)  90.5051\n(22.4811)

                   4 m               6 m              12 m  \
n
2                  NaN               NaN               NaN
3                  NaN               NaN               NaN
4                  NaN               NaN               NaN
6      0.7578\n(0.8145)               NaN               NaN
9      1.9035\n(1.3019)    0.9319\n(1.0316)               NaN
12     3.0854\n(1.7839)    2.0945\n(1.5572)               NaN
24     9.7430\n(4.4476)    7.4176\n(4.0333)    3.3375\n(2.9664)
36    19.3183\n(8.1948)   16.4220\n(7.8969)    9.7373\n(6.4471)
48   31.1264\n(10.3852)  28.7512\n(10.3842)  20.4112\n(10.0482)
60   41.3763\n(11.9820)  39.5353\n(11.8026)  33.8727\n(11.6803)
120  88.8551\n(22.5070)  85.5376\n(22.4608)  77.1479\n(22.0153)

                  24 m              60 m
n
2                  NaN               NaN
3                  NaN               NaN
4                  NaN               NaN
6                  NaN               NaN
9                  NaN               NaN
12                 NaN               NaN
24                 NaN               NaN
36     1.3963\n(3.6946)               NaN
48     5.9877\n(7.6863)               NaN
60   17.5106\n(11.2320)               NaN
120  60.3227\n(22.0579)  23.7002\n(18.3383)
```